Collection ToTAL — Réconciliation · 21 livres

L17

Guide Jaune — Clarté et spécification

Partie A

Quand les mots trahissent :

le diagnostic de la spécification floue

> *« La spécification floue est une promesse que tout le monde croit avoir entendue différemment. »*

Images et couverture par Soléane Bélanger
Léo Michel Bélanger, fondateur d'ORIGINALIUM
Collection ToTAL — Réconciliation · Livre 17 sur 21

Table des matières — Partie A

Introduction du Livre L17 — La démo qui révèle tout 5

Chapitre 1 — Le malentendu comme système 15

1.1 Pourquoi les malentendus sont structurels, pas individuels 16

1.2 La taxonomie des ambiguïtés 20

1.3 L'histoire du projet PRISM 26

1.4 L'anatomie d'un malentendu — comment il se forme 30

1.5 Pourquoi personne ne pose les questions 34

1.6 Scène d'ouverture — Émilie face à ses notes 38

1.7 Contraste DREAM-TEAM : comment ORIGINALIUM a appris 42

Chapitre 2 — Le diagnostic de spécification : lire entre les lignes 47

2.1 L'audit de spécification — outil de diagnostic 48

2.2 Les 7 mots dangereux dans une spécification 54

2.3 Le glossaire de projet comme infrastructure de clarté 59

2.4 La session d'élicitation structurée 65

2.5 Le test du taxi 70

2.6 Anecdote — Rafael et la « réconciliation des données » 74

2.7 Scène DREAM-TEAM — Théo au tableau blanc 78

Chapitre 3 — La spécification IF…THEN… : le format qui élimine les ambiguïtés .. 83

3.1 Rappel du format IF…THEN… 84

3.2 La transformation d'une spec floue en IF…THEN… précis 88

3.3 Les cas d'exception comme specs à part entière 94

3.4 Jonas et la « définition de terminé » pour chaque story 99

3.5 Le workshop IF…THEN… avec le client 103

3.6 Anecdote — Nadia et le projet sauvé 108

Chapitre 4 — La culture de la question 113

4.1 Pourquoi les équipes ne posent pas les questions 114

4.2 Le protocole d'ambiguïté 119
4.3 Émilie reprend sa voix 124
4.4 Anne comme partenaire de clarté 128
4.5 « Ce qui n'a pas fonctionné » — La première tentative 133
Conclusion de la Partie A 138

INTRODUCTION DU LIVRE L17

La démo qui révèle tout

Le jaune comme couleur de la clarté

Il y a des couleurs qui portent une signification au-delà de leur teinte. Le rouge est l'urgence. L'orange est l'action. Le vert est la fluidité. Et le jaune ? Le jaune est la lumière elle-même.

Le soleil est jaune — cette étoile qui dissipe l'obscurité et rend visible ce qui était caché. La signalisation routière est jaune quand elle veut alerter, interpeller, forcer l'attention. L'or est jaune — le métal précieux dont la valeur est reconnue universellement, qui traverse les civilisations et les siècles sans perdre sa substance. Et les post-it — ces petits rectangles jaunes qui couvrent les tableaux des équipes agiles du monde entier — sont jaunes parce qu'ils sont faits pour être vus, lus, et compris immédiatement.

Le Guide Jaune est le guide de cette lumière particulière qu'on appelle la clarté. Ce n'est pas la clarté de la simplicité naïve — ce n'est pas l'illusion que tout devrait être facile à comprendre. C'est la clarté du travail bien fait, la clarté de la précision, la clarté de l'effort consenti pour que chaque mot dans une spécification signifie exactement ce qu'il est censé signifier pour toutes les personnes qui le lisent.

Dans la collection ToTAL — Réconciliation (21 livres), chaque guide pratique adresse un défi central qui se manifeste dans les équipes de transformation numérique. Le

Guide Rouge (L15) s'attaque à l'urgence chronique. Le Guide Orange (L16) combat le chaos de l'adaptation constante. Le Guide Jaune (L17) est différent. Il s'adresse à des équipes qui peuvent livrer rapidement. Des équipes compétentes, organisées, dotées de bonnes intentions. Des équipes qui livrent — mais qui livrent souvent la mauvaise chose.

Parce qu'entre ce que le client voulait, ce que l'analyste a spécifié, et ce que le développeur a implémenté, il y a eu des traductions successives. Et dans chaque traduction, quelque chose s'est perdu. Quelque chose d'essentiel. Le sens.

Une scène universelle

Scène d'ouverture — Sprint Review — Projet Helios — Semaine 12

La salle de conférence est pleine. Nadia a disposé les chaises en arc de cercle pour que tout le monde ait une bonne vue sur le grand écran. Jonas installe son ordinateur portable, vérifie la connexion, ajuste la fenêtre du navigateur. Émilie a ses notes. Rafael parcourt ses messages sur son téléphone en attendant que ça commence.

Anne est assise face à l'écran, les bras croisés, le regard attentif. Elle a traversé la ville pour être là ce matin. Elle n'a pas l'habitude de faire le déplacement — d'habitude elle envoie un représentant. Mais ce sprint était important. Trois semaines de travail. La fonctionnalité principale du module de gestion des exceptions.

Jonas commence sa démonstration. Sa voix est calme, professionnelle. Il navigue dans l'interface, explique chaque étape. L'application s'ouvre, les données s'affichent, les boutons répondent. Tout fonctionne exactement comme la spécification l'indiquait.

Anne regarde. Elle ne dit rien pendant cinq minutes. Ses bras restent croisés.

Son expression ne change pas.

Jonas termine : « Et voilà le flux complet. La fonctionnalité est opérationnelle, les tests ont passé, la performance est dans les critères acceptables. »

Silence.

Anne : « Ce n'est pas ce que je voulais dire. »

Jonas, prudemment : « Je comprends, mais c'est exactement ce qui était décrit dans la spécification. Section 3, paragraphe 2 : "Le système affiche les exceptions dans une liste filtrée par statut." »

Anne : « Je sais. La spec ne décrivait pas ce que je voulais. »

Long silence dans la salle.

Nadia ouvre la bouche, la referme. Rafael regarde ses mains. Émilie note quelque chose dans son carnet. Jonas fixe l'écran, comme s'il espérait que l'application allait se transformer en quelque chose d'autre.

Trois semaines de travail. Le code est propre. Les tests passent. La spécification a été respectée à la lettre.

Et pourtant — ce n'était pas ça.

Cette scène — vous l'avez vécue. Peut-être du côté d'Anne, avec cette sensation de voir quelque chose qui n'est pas faux mais qui n'est pas juste. Peut-être du côté de Jonas, avec cette frustration d'avoir bien fait ce qu'on vous avait demandé et de découvrir que la demande elle-même était le problème. Peut-être du côté d'Émilie, avec cette conscience aiguë que vous aviez vu venir quelque chose — mais que vous n'aviez pas su comment l'empêcher.

Ce moment — la démo qui révèle le malentendu — est le moment fondateur du Guide Jaune. C'est le point d'arrivée de tout ce qui ne s'est pas bien passé en amont. Et c'est à partir de ce moment, de cette douleur particulière et très précise,

que nous allons construire un ensemble de pratiques, d'outils et de changements culturels qui forment ensemble une approche cohérente de la spécification vivante.

Le problème central du Guide Jaune

Dans les projets de transformation numérique, les bugs les plus coûteux ne viennent pas du code. Ils viennent des malentendus. Ils viennent de l'écart entre ce que le client voulait dire, ce que l'analyste a compris, ce que la spécification a formalisé, et ce que le développeur a implémenté.

Cet écart a un nom dans le SDLC réconcilié : **la dérive de sens**. Et contrairement à un bug de code — qui se détecte à l'exécution, qui laisse un message d'erreur, qui pointe vers une ligne précise — la dérive de sens est invisible jusqu'au moment précis où elle devient catastrophique. Elle ne génère pas d'alerte. Elle ne déclenche pas de test en échec. Elle attend, silencieuse, la démo de fin de sprint.

La dérive de sens est particulièrement insidieuse dans les organisations comme celle qu'incarne notre équipe dans ce guide : une firme conseil nationale en transformation numérique qui livre des solutions personnalisées à des clients grands comptes. Dans ces contextes, plusieurs facteurs amplifient le risque :

- **Les équipes sont mixtes.** Des consultants externes et des équipes client qui n'ont pas les mêmes codes, les mêmes vocabulaires, les mêmes cadres de référence. Ce qui est évident pour un consultant agile né dans l'univers des user stories peut être totalement abstrait pour un directeur métier formé dans la gestion de projet en V.
- **Les projets durent entre 3 et 18 mois.** Le contexte change. Les interlocuteurs changent. Les priorités changent. Une définition établie en mois 1 peut devenir caduque en mois 6 — sans que personne ne le signale formellement.
- **Le client "sait ce qu'il veut"** — mais n'a souvent jamais eu à le formaliser avec le niveau de précision qu'exige un logiciel. Dans la tête d'Anne, la

fonctionnalité est parfaitement claire. C'est quand elle devient une spec textuelle que la clarté disparaît.

- **Chaque projet commence avec une compréhension différente des mêmes mots.** "Intégration", "notification", "rapport", "validation", "workflow" — ces mots ont autant de définitions qu'il y a d'acteurs autour de la table. Et personne ne le sait jusqu'à la démo.

Le Guide Jaune est le guide qui s'attaque à cette réalité de front. Il ne promet pas d'éliminer toute ambiguïté — l'ambiguïté est inhérente au langage humain. Il promet d'en faire un objet de travail explicite plutôt qu'un problème caché, d'en réduire drastiquement l'occurrence et l'impact, et de construire dans les équipes une culture où la question posée à temps est une compétence professionnelle valorisée — pas un aveu d'ignorance.

La structure du Guide Jaune

Le Guide Jaune est organisé en deux parties complémentaires :

La Partie A — Quand les mots trahissent : le diagnostic de la spécification floue (ce volume) pose le diagnostic. Elle identifie les mécanismes qui produisent les malentendus, donne une taxonomie des ambiguïtés, et introduit les premiers outils de détection et de prévention. Elle répond à la question : *comment reconnaître qu'on a un problème de spécification ?*

La Partie B — Spécifier pour construire juste : les pratiques du SDLC réconcilié approfondit l'intégration de ces pratiques dans le cycle complet du projet, de la phase ÉLICITER à la phase CONFIRMER. Elle répond à la question : *comment construire un système de spécification qui évolue avec le projet sans perdre sa clarté ?*

Les deux parties forment un tout. Mais chacune peut être lue indépendamment selon la maturité de votre équipe. Si vous avez récemment vécu votre propre version de la scène d'ouverture, commencez par la Partie A. Si vous avez déjà

diagnostiqué le problème et cherchez les solutions systémiques, la Partie B vous attend.

Les personnages de ce guide

Le Guide Jaune suit une équipe projet tout au long de son apprentissage de la clarté. Ces personnages ne sont pas des héros parfaits — ce sont des professionnels compétents qui font face à des défis réels, qui font des erreurs, qui apprennent, et qui, progressivement, transforment leur façon de travailler.

Nadia est directrice de projet senior avec 14 ans d'expérience. Elle est brillante organisatrice, excellente communicatrice, et possède un talent naturel pour maintenir les équipes en mouvement. Son angle mort : elle croit sincèrement que les ambiguïtés "se règlent d'elles-mêmes en avançant". Pour elle, lever l'incertitude est une question d'exécution, pas de spécification. Le Guide Jaune sera son apprentissage le plus difficile — et le plus transformateur.

Rafael est architecte solution avec 11 ans d'expérience. Son travail consiste à traduire les besoins en architecture technique — et il souffre profondément quand les besoins changent en cours de route. Chaque changement tardif représente pour lui des heures de refonte, des décisions d'architecture invalidées, de la frustration accumulée. Il a développé une résilience à cette douleur qui masque une vérité : il a renoncé à demander de la clarté parce qu'il a appris que la clarté n'arrive jamais assez tôt.

Émilie est analyste d'affaires avec 5 ans d'expérience. C'est le personnage pivot du Guide Jaune. Elle possède une capacité naturelle à voir les ambiguïtés — mais pas encore la légitimité perçue ni les outils formels pour les faire remonter sans passer pour "difficile" ou "procédurière". Son évolution au fil de ces pages est le cœur de la transformation que ce guide documente.

Jonas est développeur lead avec 8 ans d'expérience. Il implémente ce qu'il comprend — et il comprend généralement très bien. Le problème : ce qu'il comprend et ce que le client voulait sont deux choses différentes avec une

fréquence alarmante. Il a développé des instincts de prévention (il pose des questions techniques précises avant de commencer) mais manque d'un cadre formel pour les systématiser.

Anne est directrice métier chez le client grand compte. Elle représente le client dans toute sa réalité : compétente dans son domaine, sachant parfaitement ce qu'elle veut d'un point de vue métier, mais n'ayant jamais eu à formaliser ce savoir avec le niveau de précision qu'exige la construction d'un logiciel. Elle n'est pas le problème — elle est la partenaire que l'équipe n'a pas encore appris à engager correctement.

En contrepoint, la **DREAM-TEAM ORIGINALIUM** — composée de Théo (PO), Rania (tech lead) et Karim (QA) — illustre à travers ses propres expériences comment les pratiques du Guide Jaune s'appliquent et produisent des résultats mesurables. Ils n'ont pas toujours su faire — eux aussi ont eu leur propre projet PRISM. Mais ils ont appris. Et leur apprentissage éclaire le chemin.

Un mot sur la couleur jaune — et ce que ce guide ne fait pas

Le Guide Jaune est analytique par nature. Plus que les autres guides de la collection, il travaille avec des outils, des formats, des grilles d'analyse et des taxonomies. Cela reflète la nature du problème qu'il adresse : la spécification floue est un problème de précision, et la précision s'apprend par la pratique de la précision.

Mais le Guide Jaune ne cherche pas à transformer les êtres humains en machines à spécifier. Il ne prétend pas qu'un format parfait élimine le besoin de conversation. Il ne propose pas un système rigide qui remplacerait le jugement par la procédure. Au contraire — chacun des outils présentés dans ce guide est conçu pour enrichir la conversation, pas pour la remplacer. La spécification IF…THEN… est un format qui pose les bonnes questions. Le glossaire de projet est un accord negocié, pas un dictionnaire imposé. Le test du taxi est un exercice d'empathie autant que de précision.

La clarté, en fin de compte, n'est pas une propriété d'un document. C'est l'état dans lequel se trouvent deux esprits humains qui ont pris le temps de se comprendre vraiment.

Ce guide vous donne les outils pour créer cet état — systématiquement, reproductiblement, à travers les projets et les équipes.

La chaîne est simple : **Clarté → qualité → confiance → excellence durable.**

Commençons par le début. Commençons par comprendre pourquoi les malentendus arrivent — pas comme des accidents, mais comme le résultat prévisible d'un système de communication inadapté.

CHAPITRE 1

Le malentendu comme système

> *« Le malentendu n'est pas un dysfonctionnement du système — il est le résultat naturel d'un système qui n'a pas été conçu pour la clarté. »*

Ce chapitre pose la thèse centrale du Guide Jaune : les malentendus dans les projets logiciels ne sont pas des accidents individuels produits par des personnes négligentes ou peu communicatives. Ce sont les résultats prévisibles et reproductibles d'un système de communication qui n'a pas été conçu pour aligner des cadres de référence différents. Comprendre cette distinction — systémique versus individuelle — est la première étape pour commencer à résoudre le problème.

Parce que si le problème est individuel, la solution est de trouver de meilleures personnes, de mieux communiquer en général, d'être plus attentif. Ces réponses sont vagues et inefficaces. Si le problème est systémique, la solution est de

concevoir un meilleur système — un système avec des formats, des processus, des artefacts et des pratiques qui rendent la clarté structurellement probable plutôt que structurellement improbable.

C'est exactement ce que fait le Guide Jaune.

1.1 Pourquoi les malentendus sont structurels, pas individuels

Commençons par une expérience de pensée. Imaginez trois personnes — Anne, Émilie et Jonas — debout devant un coucher de soleil et demandez à chacune de le décrire à quelqu'un qui ne peut pas le voir.

Anne, directrice métier qui utilise les logiciels mais ne les conçoit pas, dira peut-être : « Le ciel est rouge et orange, c'est magnifique, l'horizon est embrasé, on se sent calme. »

Émilie, analyste habituée à structurer l'information, dira peut-être : « La source lumineuse est à 15 degrés au-dessus de l'horizon, dégageant des teintes allant du rouge (700nm) à l'orange (600nm), avec des nuances de jaune et de rose dans les zones de diffusion. »

Jonas, développeur formé à la précision technique, dira peut-être : « La scène présente un gradient chromatique de #cc2200 à #ff9900 dans le tiers inférieur du champ visuel, avec des variations de luminosité entre 200 et 800 lux selon l'angle d'observation. »

Ils regardent tous les trois exactement la même chose. Leurs descriptions sont toutes exactes. Et pourtant — si vous demandiez à un artiste de peindre la scène sur la base de chacune de ces descriptions, vous obtiendriez trois tableaux très différents.

C'est le premier mécanisme structurel du malentendu : **le cadre de référence**. Chaque acteur dans un projet possède un cadre de référence construit par son

expérience, sa formation, son rôle et son vocabulaire professionnel. Ces cadres ne sont pas des défauts — ils sont des atouts. Anne comprend profondément le contexte métier. Émilie comprend profondément la structure des besoins. Jonas comprend profondément les contraintes techniques. Ces compréhensions différentes sont exactement ce qui rend l'équipe pluridisciplinaire précieuse.

Le problème n'est pas la différence des cadres de référence. Le problème est l'hypothèse implicite que ces cadres sont identiques.

Quand Anne dit "notification", elle pense à l'email qu'elle recevait dans son ancien système — un email résumant les actions en attente, envoyé tous les matins. Quand Jonas entend "notification", il pense à une notification push — une alerte instantanée sur l'interface. Ces deux réalités sont aussi légitimes l'une que l'autre. Mais si personne ne pose la question "qu'est-ce qu'une notification dans ce projet ?" — chacun continue avec son hypothèse, et la démo de fin de sprint sera la première fois que les deux définitions se confrontent.

Le deuxième mécanisme structurel du malentendu est **la chaîne de traduction**. Dans un projet typique, l'information ne va jamais directement d'Anne à Jonas. Elle passe par une série de transformations :

1. Anne exprime un besoin dans son vocabulaire métier, avec ses hypothèses implicites, en parlant à Émilie lors d'un atelier d'élicitation.
2. Émilie traduit ce besoin dans sa propre compréhension, l'organise dans la structure d'une user story ou d'un document fonctionnel, et l'écrit dans le vocabulaire de l'analyse.
3. La story est revue en équipe, peut-être ajustée, puis intégrée dans le backlog.
4. Jonas lit la story avant ou pendant le sprint. Il la traduit dans son cadre technique : quelles tables de données, quel endpoint, quelle logique applicative.
5. Le code est écrit.

À chaque étape de cette chaîne, une traduction. À chaque traduction, une opportunité de perte de sens. Et voici l'aspect le plus pernicieux : chaque traduction est faite en toute bonne foi, par des professionnels compétents, qui sont convaincus d'avoir bien compris. La perte de sens est invisible à chacun des acteurs — parce que chacun possède une version cohérente dans son propre cadre de référence.

Le troisième mécanisme structurel est **l'implicite culturel**. Dans tout contexte de travail, il existe un ensemble d'informations que tout le monde considère "évidentes" et ne juge pas nécessaire de formaliser. Ces évidences forment une culture tacite. Le problème : dans un projet mixte (consultants + équipe client), les cultures tacites des deux mondes ne sont pas identiques. Ce qui est évident pour un consultant agile — que chaque story doit avoir des critères d'acceptation explicites — peut être totalement étranger à un client qui n'a jamais travaillé en agile. Ce qui est évident pour le client — que "le système doit gérer les exceptions selon les règles de conformité" — peut cacher un monde entier de règles métier non documentées que le consultant ne connaît pas.

Ces trois mécanismes — cadres de référence, chaîne de traduction, implicites culturels — se combinent pour créer un environnement où les malentendus ne sont pas des accidents. Ce sont des résultats attendus d'un système qui n'a pas été conçu pour les prévenir.

La bonne nouvelle : un système peut être redesigné. C'est ce que fait le Guide Jaune.

Outil de réflexion — Le test des trois cadres

Avant chaque atelier d'élicitation avec un nouveau client, posez cette question à chaque membre de l'équipe projet (y compris côté client) : *« Décrivez en une phrase ce que fait ce système — comme si vous l'expliquiez à quelqu'un dans votre domaine. »*

Collectez les réponses. Comparez-les. Les divergences que vous observez sont précisément les zones d'ambiguïté à adresser en priorité avant que le

développement commence.

1.2 La taxonomie des ambiguïtés — outil terrain

Pour travailler sur les ambiguïtés, il faut d'abord les nommer. Les nommer avec précision, de façon à ce que tout le monde dans l'équipe comprenne de quel type d'ambiguïté on parle et comment l'adresser. C'est l'objectif de la taxonomie des ambiguïtés du Guide Jaune.

Cette taxonomie identifie cinq types fondamentaux d'ambiguïtés qui se retrouvent dans toutes les spécifications de projets logiciels. Chaque type a ses propres caractéristiques, ses propres signaux d'alerte, et ses propres stratégies de résolution.

Type 1 — L'ambiguïté lexicale

L'ambiguïté lexicale est la plus connue et paradoxalement la moins bien adressée. Elle se produit quand le même mot a plusieurs sens légitimes selon le cadre de référence de celui qui le lit. Le mot est "clair" — mais sa signification ne l'est pas.

Dans le projet Helios, le mot "notification" est un cas d'école d'ambiguïté lexicale. Ce mot a au moins six définitions techniques courantes dans le monde du logiciel : email, SMS, notification push mobile, alerte in-app, message dans un centre de notifications, ou webhook vers un système tiers. Pour Anne, habitituée à recevoir des emails de son ERP, "notification" est évidemment un email. Pour Jonas, développeur habitué aux applications mobiles, "notification" est évidemment une alerte push. Pour Rafael, architecte qui pense aux intégrations, "notification" peut être un webhook. Personne n'a tort. Tout le monde a raison dans son cadre. Et personne ne le sait.

Autres exemples d'ambiguïtés lexicales fréquentes dans les projets :

Mot	Définition possible A	Définition possible B	Définition possible C
Rapport	Tableau de bord interactif	Export Excel généré à la demande	Email hebdomadaire automatique
Validation	Vérification automatique des données	Approbation manuelle par un superviseur	Confirmation par l'utilisateur final
Intégration	Synchronisation en temps réel	Import/export par fichier CSV	Accès direct à la base de données
Tableau de bord	Vue synthétique avec KPIs graphiques	Liste filtrée d'enregistrements	Page d'accueil personnalisable
Accès	Droit de lecture uniquement	Droit de lecture et écriture	Accès conditionnel selon le rôle
Workflow	Processus manuel avec étapes guidées	Automatisation complète de bout en bout	Approbation séquentielle multi-niveaux

Stratégie de résolution : Le glossaire de projet (présenté en section 2.3) est l'outil principal pour adresser les ambiguïtés lexicales. Chaque terme clé du projet reçoit une définition négociée, validée par toutes les parties, et référencée dans toutes les spécifications.

Type 2 — L'ambiguïté structurelle

L'ambiguïté structurelle se produit quand une phrase peut être interprétée de plusieurs façons grammaticalement valides. La phrase est correctement construite — mais sa structure syntaxique permet plusieurs lectures.

Exemple classique : *"Les utilisateurs peuvent modifier leurs préférences."*

Cette phrase semble parfaitement claire. Mais elle cache plusieurs ambiguïtés structurelles :

- **Qui sont "les utilisateurs" ?** Tous les utilisateurs ? Seulement les administrateurs ? Les utilisateurs avec un certain niveau d'abonnement ?

- **Que signifie "leurs" ?** Leurs propres préférences uniquement ? Ou peuvent-ils modifier les préférences d'autres utilisateurs ?
- **Que signifie "modifier" ?** Lecture et écriture ? Ou seulement ajuster certains champs prédéfinis ?
- **Quelles sont "leurs préférences" exactement ?** La langue d'interface ? Les notifications ? La disposition des colonnes dans les tableaux ? L'heure de réception des rapports ?
- **"Peuvent" implique-t-il que c'est optionnel ou que c'est une fonctionnalité qui doit être disponible ?**

Cinq interprétations légitimes dans une seule phrase de six mots. Et chacun dans l'équipe va choisir l'interprétation qui lui semble "la plus logique" — c'est-à-dire celle qui correspond à son cadre de référence.

Stratégie de résolution : La transformation IF…THEN… (chapitre 3) décompose chaque phrase structurellement ambiguë en clauses précises qui spécifient l'acteur, l'action, les conditions et les résultats attendus.

Type 3 — L'ambiguïté de portée

L'ambiguïté de portée se produit quand on ne sait pas ce qui est inclus dans le périmètre d'une fonctionnalité et ce qui ne l'est pas. La spécification décrit quelque chose — mais ne dit pas où ce quelque chose commence et où il finit.

Exemple : *"Un module de reporting."*

Cette formulation ouvre une infinité de questions de portée :

- Reporting sur quelles données ? Toutes les données du système ? Seulement certains modules ?
- Pour qui ? Tous les utilisateurs ? Seulement les managers ? Seulement le département finance ?
- Avec quelles données sources ? En temps réel ? Sur des snapshots ? Sur des données archivées ?
- Avec quels formats d'export ? PDF ? Excel ? CSV ? Aucun ?

- Avec quelles capacités de personnalisation ? L'utilisateur peut-il créer ses propres rapports ? Ou seulement consulter des rapports prédéfinis ?
- Avec quelle fréquence ? Rapports à la demande ? Rapports automatiques planifiés ?

L'ambiguïté de portée est particulièrement dangereuse parce qu'elle affecte l'estimation de l'effort. "Un module de reporting" peut représenter 2 semaines ou 6 mois de développement selon ce qui est inclus dans la portée. Et si la portée n'est pas définie explicitement, elle sera définie implicitement — par les hypothèses de chacun.

Stratégie de résolution : L'audit de spécification (section 2.1) inclut une vérification systématique de la portée pour chaque fonctionnalité. La session d'élicitation structurée (section 2.4) pose explicitement les questions de périmètre.

Type 4 — L'ambiguïté temporelle

L'ambiguïté temporelle se produit quand on ne sait pas *quand* quelque chose doit se produire. Les conditions de timing sont omises ou formulées de façon vague.

Exemple : *"Le système doit envoyer une confirmation."*

Questions de timing :

- Immédiatement après l'action de l'utilisateur ?
- Dans les 5 minutes ?
- Dans les 24 heures ?
- Après une validation manuelle par un superviseur ?
- Au prochain cycle de batch (toutes les heures ? toutes les nuits ?) ?
- Après la synchronisation avec le système tiers ?

Pour Anne, "le système envoie une confirmation" signifie probablement "immédiatement, comme quand je commande sur un site e-commerce". Pour Rafael, "confirmation" dans le contexte d'un système d'entreprise avec une architecture batch signifie peut-être "dans le prochain cycle de traitement". Pour Jonas, ça dépend de l'implémentation — qu'est-ce qu'il y a dans la spec ?

Les ambiguïtés temporelles sont fréquemment sources de problèmes d'architecture. Si "temps réel" signifie quelque chose de différent pour le client et pour l'équipe technique (voir anecdote section 3.6), l'architecture choisie peut être fondamentalement inadaptée aux attentes réelles.

Stratégie de résolution : Chaque spécification temporelle doit être exprimée en unités de temps mesurables (secondes, minutes, heures). Les ambiguïtés "temps réel" et "rapide" sont particulièrement dangereuses et doivent être traquées systématiquement (section 2.2).

Type 5 — L'ambiguïté conditionnelle

L'ambiguïté conditionnelle est la plus souvent sous-estimée et la plus coûteuse. Elle se produit quand les cas d'exception ne sont pas spécifiés. La spec décrit ce qui se passe dans le cas nominal — mais ne dit pas ce qui se passe quand les choses ne se passent pas comme prévu.

Exemple : *"L'utilisateur soumet le formulaire et le système enregistre les données."*

Cas d'exception non spécifiés :

- Que se passe-t-il si la connexion réseau est perdue pendant la soumission ?
- Que se passe-t-il si un champ obligatoire est manquant ?
- Que se passe-t-il si l'utilisateur ferme l'onglet avant de soumettre ?
- Que se passe-t-il si le serveur est temporairement indisponible ?
- Que se passe-t-il si les données violeront une règle de validation métier ?
- Que se passe-t-il si la session a expiré entre le chargement du formulaire et la soumission ?
- Que se passe-t-il si l'utilisateur soumet le même formulaire deux fois (double-clic) ?

Dans la pratique, le cas nominal représente environ 20% du comportement réel d'un système. Les 80% restants sont des cas d'exception, des conditions limites, des situations d'erreur. Et une spec qui ne couvre que le cas nominal est une spec à

20% de complétude — même si elle semble complète à qui la lit pour la première fois.

Stratégie de résolution : La session d'élicitation structurée (section 2.4) inclut systématiquement une étape dédiée aux cas d'exception. Le format IF…THEN… (chapitre 3) crée un cadre naturel pour spécifier le comportement exceptionnel.

Outil terrain — La grille de détection rapide des 5 types

À utiliser lors de la revue d'une spécification :

Type d'ambiguïté	Signal d'alerte dans le texte	Question à poser
Lexicale	Terme non défini dans le glossaire	« Qu'est-ce que ce mot signifie précisément dans ce projet ? »
Structurelle	Sujet vague ("les utilisateurs"), verbe modal ("peuvent", "doivent")	« Qui exactement ? Toujours ? Dans quelles conditions ? »
De portée	Terme générique ("module", "système", "rapport")	« Qu'est-ce qui est inclus ? Qu'est-ce qui est exclu ? »
Temporelle	Adverbe ("immédiatement", "rapidement", "dès que")	« Dans quel délai précis, en quelle unité de temps ? »

Conditionnelle

Absence de mention d'erreurs ou d'exceptions

« Que se passe-t-il si ça ne marche pas comme prévu ? »

1.3 L'histoire du projet PRISM — anecdote terrain

Avant d'aller plus loin dans les outils et les techniques, il faut ancrer la compréhension du problème dans une réalité concrète. Le projet PRISM est cette réalité. Il n'est pas présenté comme un exemple exceptionnel de dysfonctionnement — au contraire. C'est un projet parfaitement ordinaire, géré par des personnes compétentes, avec de bonnes intentions, qui a produit exactement ce qu'il devait produire — et qui a néanmoins raté sa cible.

Contexte du projet PRISM — Reconstruction narrative

L'équipe reçoit le mandat en début d'année : un grand compte dans le secteur financier a besoin d'un système de reporting intégré pour ses équipes de conformité. Le budget est approuvé, l'équipe est constituée, le projet démarre.

La spécification initiale tient en une phrase-clé : *« Un système de reporting intégré pour les équipes de conformité, permettant de consolider les données de plusieurs sources et de générer des rapports standardisés selon les exigences réglementaires. »*

Nadia a lu cette phrase et a hoché la tête. Rafael a commencé à dessiner l'architecture de consolidation des données. Émilie a travaillé avec le contact client — pas Anne directement, mais son assistant — pour détailler les formats de rapports. Jonas a commencé le développement en semaine 6.

Neuf mois de travail. Une équipe expérimentée. Un processus agile avec des sprints bimensuels, des reviews régulières, des démos aux partenaires projet. Un budget respecté. Un calendrier tenu.

La livraison finale : un système de reporting impressionnant. Des tableaux de bord interactifs. Des exports PDF aux normes réglementaires. Des filtres multi-dimensionnels sur toutes les sources de données. Une performance de génération de rapport en moins de 3 secondes pour des jeux de données de 500 000 lignes. Techniquement irréprochable.

Démo finale — Salle de direction — Semaine 38

Anne est là. C'est la première fois qu'elle voit le système complet. Elle a reçu des rapports d'avancement tout au long du projet, mais n'avait pas participé directement aux reviews de sprint — son agenda ne le permettait pas, et son assistant avait validé à sa place.

La démo se passe bien. Le système est impressionnant. Anne pose des questions pertinentes, s'intéresse aux détails techniques, apprécie la qualité de l'interface.

Puis, après 45 minutes de démonstration : « C'est très bien. Mais… ce dont j'avais besoin, c'est d'un système d'alertes qui déclenche un workflow d'approbation quand un rapport dépasse un seuil. Le reporting, c'était juste pour illustrer le contexte dans ma demande initiale. »

Silence.

Nadia : « Un workflow d'approbation… »

Anne : « Oui. Quand un indicateur de conformité dépasse le seuil réglementaire, le système doit alerter le responsable de conformité, qui doit approuver ou escalader dans les 48 heures. C'est le cœur du besoin. Le reporting que vous avez construit est utile — mais ce n'est pas ce qui me pose

un problème aujourd'hui. »

Nadia calcule mentalement : 9 mois. Le mot "reporting" avait été compris différemment par tout le monde. Le mot "intégré" avait été compris comme "consolidation des données" plutôt que "intégré dans un workflow de conformité". Et personne n'avait posé la question fondamentale : « Qu'est-ce que ce système fait pour vous que vous ne pouvez pas faire aujourd'hui ? »

L'histoire du projet PRISM n'est pas une histoire de mauvaises personnes. C'est une histoire de mots. Quatre mots : "reporting intégré pour la conformité". Ces quatre mots ont été utilisés dans la spécification initiale, validés par toutes les parties, et ont orienté neuf mois de travail dans la mauvaise direction — non pas parce qu'ils étaient faux, mais parce qu'ils n'avaient pas le même sens pour tout le monde.

Pour Anne : un système qui détecte les dépassements de seuil et déclenche des actions. Le "reporting" était métaphorique — elle voulait dire "visibilité sur la conformité".

Pour Nadia : un système qui produit des rapports réglementaires de façon efficace. Le mot "reporting" était littéral.

Pour Rafael : une architecture de consolidation et de présentation de données. "Intégré" signifiait "toutes les sources de données dans un seul système".

Pour Émilie : des formats de rapports standardisés aux normes réglementaires. "Conformité" signifiait "format conforme aux exigences".

Pour Jonas : des queries complexes avec des exports formatés. "Reporting" était un domaine technique bien maîtrisé.

Cinq personnes. Cinq compréhensions. Une seule phrase dans la spécification. Et neuf mois de travail qui produisent un résultat excellent — pour le mauvais problème.

Le projet PRISM n'est pas exceptionnel. Des variantes de cette histoire se produisent dans des dizaines d'organisations chaque année. Ce qui est exceptionnel, c'est la décision prise après : ne plus jamais laisser quatre mots décider du sens d'un projet de neuf mois.

> **⚠️ Signal de vigilance — Le "résumé exécutif" comme spécification**
>
> Le projet PRISM a commencé avec une phrase-résumé comme spécification fonctionnelle. Ce pattern est extrêmement courant — et extrêmement risqué. Une phrase-résumé est un point de départ pour une conversation, pas un point d'arrivée pour une spécification. Si votre projet est en train de démarrer sur la base d'un paragraphe d'une page ou moins, c'est un signal d'alarme qui demande une session d'élicitation approfondie avant que le développement commence.

1.4 L'anatomie d'un malentendu — comment il se forme et pourquoi personne ne le voit

Le projet PRISM nous donne le matériau pour disséquer l'anatomie d'un malentendu. Comprendre exactement comment un malentendu se forme — étape par étape — est essentiel pour savoir où intervenir pour le prévenir.

Un malentendu majeur dans un projet logiciel se forme en cinq étapes distinctes. Chaque étape est une opportunité d'intervention — et une opportunité manquée.

Étape 1 — L'expression originelle

Anne dit quelque chose. Elle utilise son vocabulaire métier, avec ses hypothèses implicites, dans son cadre de référence. Elle ne ment pas, elle n'est pas imprécise intentionnellement — elle communique de façon parfaitement normale, comme elle le fait avec ses collègues depuis des années. Dans son équipe, tout le monde sait ce

que "système de reporting intégré pour la conformité" signifie — parce qu'ils partagent le même contexte implicite.

Opportunité manquée n°1 : demander à Anne de décrire non pas ce qu'elle veut que le système fasse, mais ce que le système fera pour elle que son système actuel ne fait pas, et quel problème concret sera résolu. Cette question aurait probablement révélé l'existence du workflow d'approbation dès le premier jour.

Étape 2 — La première traduction

Émilie entend la demande d'Anne et la traduit dans sa propre compréhension. Elle note les mots clés, identifie les domaines fonctionnels, structure les besoins dans le format qui lui est familier. Elle est convaincue d'avoir bien compris — parce que sa traduction est cohérente dans son propre cadre de référence. Ce qu'elle ne sait pas, c'est qu'elle a silencieusement éliminé les parties de la demande d'Anne qui n'entraient pas facilement dans son cadre analytique.

Opportunité manquée n°2 : Émilie aurait pu relire sa compréhension à Anne à voix haute avant de la formaliser : "Si j'ai bien compris, vous avez besoin de X, Y et Z. Est-ce que c'est exact ? Est-ce qu'il manque quelque chose d'important ?" Cette validation de compréhension est une pratique simple qui détecte la majorité des pertes de sens à l'étape 2.

Étape 3 — La formalisation écrite

Émilie écrit la spécification. L'écriture impose des choix — il faut décider quels détails inclure, quels détails omettre, quel niveau de granularité est approprié. Ces choix reflètent son propre jugement sur ce qui est "évident" et ce qui ne l'est pas. Ce qui est évident pour Émilie peut être une hypothèse non partagée pour Jonas.

Opportunité manquée n°3 : une revue de spécification avec Jonas et Rafael avant que la spec soit finalisée. Cette revue n'est pas une revue technique — c'est une revue de compréhension. Chacun exprime comment il comprend la spec. Les divergences sont détectées et retournées à Émilie pour clarification.

Étape 4 — La deuxième traduction

Jonas lit la spec. Il la traduit dans son cadre technique : tables de données, API, logique applicative, performance. Cette traduction est aussi faite en bonne foi, avec toute son expérience et sa compétence. Et elle introduit un nouveau couche d'hypothèses implicites — les hypothèses techniques.

Opportunité manquée n°4 : Jonas aurait pu écrire sa compréhension technique de la spec sous forme de plan d'implémentation et le partager avec Émilie avant de commencer le développement. Cette pratique — le "plan de compréhension" — est systématique dans les équipes qui ont appris à gérer la dérive de sens.

Étape 5 — L'implémentation et le gel du sens

Le code est écrit. À partir de ce moment, la compréhension de Jonas est figée dans le comportement du système. Et le comportement du système sera révélé pour la première fois à Anne lors de la démo — pour le meilleur ou pour le pire.

Opportunité manquée n°5 : des démos intermédiaires à Anne — pas juste à son assistant — à intervalles réguliers pendant le développement. Chaque démo est une opportunité de détecter la dérive de sens avant qu'elle soit intégralement implémentée.

Ce que cette anatomie révèle : le malentendu final n'est pas le résultat d'une seule erreur. C'est l'accumulation de cinq opportunités d'intervention manquées. Et aucune de ces opportunités manquées n'est due à de la mauvaise volonté ou à de l'incompétence. Elles sont manquées parce qu'il n'y a pas de système en place qui les rende systématiques.

Le Guide Jaune est ce système.

1.5 Pourquoi personne ne pose les questions

Si les malentendus sont si visibles rétrospectivemen — si chacun peut les voir après la démo catastrophique — pourquoi personne ne les signale avant ? Pourquoi

Émilie, qui avait noté 12 ambiguïtés en lisant la spec initiale, ne les avait-elle pas toutes remontées ?

La réponse est culturelle. Et elle est plus complexe qu'il n'y paraît.

La peur de l'incompétence perçue

Dans un milieu professionnel, poser une question est un acte socialement risqué. Poser une question signale potentiellement qu'on ne sait pas — et ne pas savoir peut être perçu comme une lacune dans un contexte où tout le monde est censé être expert. Cette peur est particulièrement forte pour Émilie, 5 ans d'expérience dans une équipe de seniors avec 8 à 14 ans d'expérience. Demander "que signifie exactement notification dans ce projet ?" risque de la faire paraître comme quelqu'un qui "ne comprend pas vite".

Cette peur est irrationnelle — poser des questions de précision est une marque de professionnalisme — mais elle est réelle et puissante. Et dans l'absence d'une culture explicite qui valorise la question, la peur l'emporte.

La culture du "on avance"

Dans de nombreuses équipes projet, la valeur implicitement dominante est la vélocité. On avance. On livre. On ne ralentit pas pour des questions qui "se verront bien en développement". Cette culture est souvent portée par le chef de projet — ici, Nadia — qui a appris à son expérience que les projets qui s'enlisent dans les détails en début de phase ne livrent jamais.

Cette intuition n'est pas fausse — il existe un vrai danger de paralysie par l'analyse. Mais elle crée une culture où lever une ambiguïté est perçu comme "ralentir l'avancement" — et donc comme quelque chose à éviter. Le coût de cette culture n'est pas visible en début de projet. Il se matérialise à la démo de la semaine 12.

L'hypothèse que "ça se verra"

Jonas voit l'ambiguïté dans "le système doit envoyer une confirmation" — mais ne pose pas la question parce qu'il se dit : "Je vais faire une implémentation raisonnable, et si ce n'est pas exactement ce qu'ils voulaient, on ajustera." Cette

hypothèse — que les ajustements tardifs sont peu coûteux — est une des illusions les plus répandues dans le développement logiciel.

En réalité, le coût d'une correction croît exponentiellement avec le temps. Une ambiguïté résolue en phase d'élicitation coûte une heure de conversation. Résolue en phase de planification : une journée de révision des specs. Résolue en phase d'implémentation : une semaine de refactoring. Résolue en phase de diffusion : un mois de correctifs et de tests. Résolue après la mise en production : plusieurs mois de gestion d'incidents et de re-développement.

L'absence de processus formel

Il n'y a pas de canal formel pour remonter les ambiguïtés. Il n'y a pas de registre d'ambiguïtés. Il n'y a pas de réunion dédiée à la résolution des ambiguïtés avant le sprint planning. L'ambiguïté qu'Émilie a notée dans son carnet n'a nulle part où aller — sauf dans son carnet, où elle reste jusqu'à la démo.

L'absence de processus transforme la gestion des ambiguïtés en initiative individuelle — ce qui signifie qu'elle dépend de la personnalité, de l'énergie et du courage de chaque individu. Ce n'est pas un système. C'est une loterie.

La peur de ralentir le projet

Même quand Émilie ose poser la question, elle anticipe la réaction de Nadia : "On n'a pas le temps pour ça. On verra en développement." Cette réaction n'est pas imaginaire — elle l'a vécue. La peur de la réaction dissuasive est aussi puissante que la peur de l'incompétence perçue.

Il faut donc non seulement créer un processus pour remonter les ambiguïtés, mais aussi changer la culture managériale qui récompense la vélocité et pénalise la précaution. Nadia devra faire cette évolution — et nous la verrons l'accomplir dans le chapitre 4.

Ce qui n'a pas fonctionné — La réunion de "pré-clarification" de Nadia

Après le projet PRISM, Nadia avait tenté d'introduire une "réunion de

clarification" hebdomadaire de 30 minutes avant le sprint planning. L'idée était bonne, l'intention était bonne — mais la réunion s'était rapidement transformée en une session où chacun défendait sa propre compréhension sans méthode de résolution structurée. Les ambiguïtés étaient soulevées, débattues — et souvent laissées non résolues faute de données pour trancher. Ce n'est pas la réunion qui manquait. C'est le *protocole* de la réunion.

Leçon : une réunion sans protocole est une conversation sans résolution. Les outils présentés dans les chapitres 2 et 3 donnent ce protocole.

1.6 Scène d'ouverture — Émilie face à ses notes

Bureau d'Émilie — Vendredi soir — Après la démo catastrophique du projet PRISM

Émilie n'est pas rentrée chez elle. Elle est encore au bureau à 19h. Son carnet de notes du projet PRISM est ouvert devant elle. Elle relit, méthodiquement, chaque page depuis le début du projet.

Elle a un stylo rouge. Elle coche. Elle compte.

Page 3 : *"'Reporting intégré' — à clarifier : intégration avec quels systèmes ?"* Coche rouge.

Page 7 : *"'Équipes de conformité' — combien d'utilisateurs ? Quels rôles ?"* Coche rouge.

Page 11 : *"'Rapports standardisés' — standardisés selon quelles normes réglementaires spécifiques ?"* Coche rouge.

Page 15 : *"'Consolider les données de plusieurs sources' — quelles sources*

exactement ? Les mêmes pour tous les rapports ?" Coche rouge.

Page 22 : *"Aucune mention d'alertes ou de workflows. À vérifier avec Anne directement."* Coche rouge. Deux étoiles.

Elle continue. Page après page. Question après question. Coche après coche.

Quand elle a terminé, elle compte : douze coches rouges. Douze ambiguïtés qu'elle avait détectées en lisant la spec initiale, lors des réunions d'élicitation, pendant les reviews de sprint. Douze questions qu'elle avait notées dans son carnet — et qu'elle n'avait pas posées à voix haute.

Elle relit la coche de la page 22. "Aucune mention d'alertes ou de workflows. À vérifier avec Anne directement." Elle avait même identifié la lacune centrale. Elle avait même noté qu'il fallait aller directement à Anne — pas à son assistant. Elle avait seulement... oublié de le faire. Ou plutôt : elle avait trouvé des raisons de le reporter. La réunion de la semaine suivante. Le sprint qui commençait. Nadia qui avait besoin d'elle sur autre chose. La peur de paraître difficile.

Douze ambiguïtés. Douze occasions. Toutes manquées.

Et les douze s'étaient retrouvées à la démo sous la même forme : « Ce n'est pas ce que je voulais dire. »

Émilie ferme son carnet. Elle prend une nouvelle feuille blanche. En haut, elle écrit : *"Protocole personnel — à partir de maintenant."*

Puis, en dessous, deux lignes :

"Chaque ambiguïté détectée est notée ET remontée. Pas plus tard. Maintenant."

"Je préfère paraître difficile que voir un an de travail livré à côté de la cible."

Elle ne sait pas encore qu'elle vient de poser la première pierre du protocole d'ambiguïté qui va transformer son équipe.

La scène d'Émilie face à ses notes n'est pas une anecdote sur l'échec. C'est une anecdote sur la prise de conscience — ce moment précis où quelqu'un comprend non seulement ce qui s'est passé, mais aussi pourquoi, et décide de ne plus jamais laisser la même chose arriver.

Cette prise de conscience est le point de départ de toute transformation. Elle ne vient pas d'une formation, d'un processus imposé, d'une directive managériale. Elle vient de la douleur de voir douze notes dans un carnet et de comprendre leur signification collective. Elle vient de l'honnêteté de se dire : "Je savais. Et je n'ai pas agi."

La transformation d'Émilie au fil de ce guide — de l'analyste qui voit les ambiguïtés mais ne les remonte pas à l'analyste qui les fait remonter systématiquement et avec une méthode claire — est le cœur de la Partie A. Parce que changer un individu est le début. Changer un système est l'objectif.

1.7 Contraste DREAM-TEAM — Comment ORIGINALIUM a appris à tuer les ambiguïtés

ORIGINALIUM — Session de rétrospective — Projet NEXUS (référence interne)

Théo pose son marker sur le tableau blanc. Il vient de finir de lister les 8 "moments de vérité" du projet NEXUS — les moments où une ambiguïté non détectée avait produit une correction coûteuse.

Rania : « On a le même pattern dans chacun de ces moments. On avait l'info. Quelqu'un avait vu quelque chose. Et on n'avait pas de canal pour le formaliser. »

Karim : « Le pire c'est le point 3. J'avais écrit dans mes notes de test : "à confirmer — le comportement attendu en cas de timeout n'est pas spécifié." Et

je l'ai mis dans un coin parce que je pensais que quelqu'un d'autre s'en occuperait. »

Théo : « Personne ne s'en est occupé. Parce que personne ne savait que toi tu l'avais vu. »

Long silence.

Rania : « Alors qu'est-ce qu'on change ? »

Théo : « Deux choses. Un registre d'ambiguïtés visible de toute l'équipe — pas dans nos carnets personnels. Et un format de spec qui rend les ambiguïtés impossibles à ignorer. »

Karim : « Le format IF…THEN… que tu nous avais montré la semaine dernière ? »

Théo : « Exactement. »

ORIGINALIUM n'a pas trouvé les réponses dans un livre — ils les ont construites à partir de leur propre projet NEXUS, leur équivalent du projet PRISM. Six mois de travail livré à côté de la cible. Une douleur suffisamment intense pour justifier un changement profond de méthode.

Ce changement s'est matérialisé en deux décisions concrètes :

Décision 1 : Chaque ambiguïté est un item visible. L'équipe ORIGINALIUM a intégré une colonne "Ambiguïté" dans son backlog. Chaque ambiguïté détectée par n'importe quel membre de l'équipe y est logguée immédiatement, avec un owner et une deadline. Une ambiguïté non résolue à la veille du sprint planning bloque automatiquement la story concernée. Il n'y a pas de dérogation possible — l'ambiguïté est résolue avant que le développement commence, ou la story ne rentre pas dans le sprint.

Décision 2 : Chaque story en format IF…THEN… avant le sprint. Théo a introduit la pratique systématique de reformuler chaque story en format IF…THEN…

lors du sprint planning. Cette reformulation prend 5 à 10 minutes par story — et elle révèle immédiatement les ambiguïtés qui subsistent. Si la story ne peut pas être reformulée en IF…THEN… sans hésitation sur les acteurs, les conditions ou les résultats attendus, elle est marquée "non prête" et retourne à la phase de définition.

Théo explique l'impact six mois après l'introduction de ces deux pratiques :

Théo — Bilan à 6 mois

« La première chose qu'on a remarquée, c'est la réduction des "re-questions en cours de sprint". Avant, Jonas venait me voir deux ou trois fois par sprint pour des clarifications en cours de développement. Maintenant, c'est peut-être une fois tous les deux sprints. »

« La deuxième chose, c'est la qualité des démos. On a arrêté d'avoir des démos où le client dit "ce n'est pas ce que je voulais". Pas complètement — il arrive encore qu'on ait raté quelque chose. Mais ce sont maintenant des petits ajustements, pas des reconstructions complètes. »

« La troisième chose — et c'est peut-être la plus importante — c'est que Karim pose maintenant ses questions sans hésiter. Il n'a plus peur de paraître difficile. Il a compris que sa question est un service rendu à l'équipe entière. »

Rania ajoute : « La question que je préfère dans la check-list : "Que se passe-t-il si l'utilisateur fait exactement le contraire de ce qu'on attend ?" Cette question révèle des dizaines de cas d'exception que personne n'avait pensé à spécifier. »

Karim sourit : « Moi c'est : "Comment saurons-nous que c'est terminé ?" Si on ne peut pas répondre en 30 secondes, la story n'est pas prête. »

La transformation d'ORIGINALIUM n'a pas été instantanée. Elle a demandé plusieurs sprints d'adaptation, des résistances, des moments de doute. Théo raconte que les deux premières semaines après l'introduction du format IF…THEN… avaient généré

des tensions — certains membres de l'équipe trouvaient le processus "trop formel", "trop lent". Le retournement avait eu lieu lors de la première démo client qui s'était passée sans "ce n'est pas ce que je voulais dire". Ce succès avait suffi à convaincre les sceptiques.

La leçon d'ORIGINALIUM pour notre équipe — Nadia, Rafael, Émilie, Jonas et Anne — est précise : les pratiques du Guide Jaune fonctionnent. Elles ont un coût en temps en amont. Et elles évitent des coûts exponentiellement plus élevés en aval. L'investissement est réel. Le retour sur investissement est mesurable.

📌 À retenir — Chapitre 1

- Les malentendus sont **structurels**, pas individuels. Ils résultent de cadres de référence différents, de chaînes de traduction, et d'implicites culturels non alignés.
- La **taxonomie des 5 types d'ambiguïtés** (lexicale, structurelle, de portée, temporelle, conditionnelle) donne un langage commun pour identifier et adresser les malentendus avant qu'ils se cristallisent dans le code.
- Le **projet PRISM** illustre comment quatre mots peuvent orienter neuf mois de travail dans la mauvaise direction — non par malveillance, mais par absence de validation systématique de la compréhension partagée.
- La **culture du silence** — peur de l'incompétence perçue, culture du "on avance", absence de processus formel — est le terreau des malentendus. La changer est un prérequis aux outils.
- La transformation commence par une **prise de conscience individuelle** (Émilie face à ses notes) et se complète par un **changement systémique** (les pratiques d'ORIGINALIUM).

🔍 Symptômes à observer dans votre équipe

- Les démos client produisent régulièrement des "ce n'est pas ce que je voulais dire" — même pour des fonctionnalités développées selon la spec.
- Les termes clés du projet (notification, rapport, workflow, validation) n'ont jamais été formellement définis et semblent signifier des choses différentes selon les personnes.
- Les membres de l'équipe hésitent à poser des questions de clarification en réunion, attendant de voir "comment ça se passe en développement".
- Les corrections post-démo sont fréquentes et portent sur des malentendus de compréhension plutôt que sur des bugs techniques.
- Les specs initiales tiennent en un ou deux paragraphes et servent de base directe au développement sans phase de raffinement.
- La relation avec le client est principalement relayée par un intermédiaire — l'équipe n'a pas accès direct aux décideurs métier.

💬 Questions de réflexion — Chapitre 1

1. Pensez à votre dernier projet significatif. Combien d'ambiguïtés avez-vous personnellement détectées et non remontées ? Pourquoi ne les avez-vous pas remontées ?
2. Dans votre équipe, quel type d'ambiguïté est le plus fréquent — lexicale, structurelle, de portée, temporelle ou conditionnelle ? Qu'est-ce que cela révèle sur votre système de spécification actuel ?
3. Si vous deviez faire l'exercice d'Émilie (relire les specs d'un projet récent avec un stylo rouge), combien de coches trouveriez-vous ? Que feriez-vous différemment ?

4. Quelle est la culture dominante dans votre équipe concernant les questions de clarification — sont-elles valorisées ou perçues comme des ralentissements ?

5. Qui dans votre équipe a le rôle explicite de détecter et de faire remonter les ambiguïtés ? Si personne, qui devrait l'avoir ?

⚠️ **Signal de vigilance — Chapitre 1**

Le signal le plus fort d'un système de spécification défaillant : si votre équipe peut livrer une fonctionnalité techniquement irréprochable, conforme à la spécification écrite, dans les délais et le budget — et que le client dit quand même "ce n'est pas ce que je voulais dire", alors le problème n'est pas dans l'exécution. Il est dans le système de spécification. Et un système de spécification défaillant ne se corrige pas en travaillant plus dur. Il se corrige en travaillant différemment — avec les outils des chapitres 2, 3 et 4.

CHAPITRE 2

Le diagnostic de spécification : lire entre les lignes

> *« Une spécification qu'on ne peut pas tester n'est pas une spécification. C'est une intention. Et les intentions ne compilent pas. »*

Le diagnostic est le premier acte du changement. Avant d'introduire de nouveaux outils, de nouveaux formats, de nouvelles pratiques, il faut savoir exactement où en est le niveau de clarté d'une spécification existante. Il faut savoir lire entre les lignes — identifier ce qui est dit, ce qui n'est pas dit, ce qui est dit mais peut être interprété de plusieurs façons.

Ce chapitre donne les outils de ce diagnostic : l'audit de spécification en 4 dimensions, la liste des 7 mots dangereux, le glossaire de projet comme infrastructure de clarté, la session d'élicitation structurée, le test du taxi, et deux anecdotes terrain qui illustrent l'application concrète de ces outils.

L'objectif de ce chapitre n'est pas de rendre les specs parfaites — il est de rendre les specs honnêtes. Une spec honnête dit clairement ce qu'elle couvre et ce qu'elle ne couvre pas. Elle identifie ses propres lacunes. Elle donne à l'équipe de développement le matériau pour construire ce qui est attendu — pas ce qui semblait raisonnable de construire.

2.1 L'audit de spécification — outil de diagnostic

Le jour de la réunion de lancement du projet Helios — le projet qui suit le projet PRISM dans la trajectoire de notre équipe — Nadia décide de faire quelque chose qu'elle n'avait jamais fait systématiquement auparavant : auditer la spécification avant que le développement commence.

Elle convoque Émilie. Elle ouvre le document de 47 pages que le client a fourni. Et elle pose une question simple : "Avant qu'on aille plus loin — est-ce qu'on peut évaluer la qualité de ce document ?"

Émilie a souri. Elle avait attendu cette question depuis deux ans.

L'audit de spécification qu'elles utilisent ensemble s'organise en quatre dimensions. Chaque dimension peut être évaluée sur une échelle de 1 à 5, et l'ensemble produit un score de maturité de la spécification qui oriente les priorités de clarification.

Dimension 1 — Complétude

La dimension de complétude évalue si la spécification couvre l'ensemble du comportement attendu du système — pas seulement les cas nominaux.

Pour chaque fonctionnalité décrite, les questions de complétude sont :

- Y a-t-il des critères d'acceptation explicites ? (Pas "le système envoie une confirmation" mais "le système envoie un email de confirmation à l'adresse X dans les Y minutes")
- Y a-t-il au moins un cas de succès ET un cas d'échec décrits ?
- Les conditions limites sont-elles spécifiées ? (Que se passe-t-il à la valeur maximale ? À la valeur minimale ? Avec zéro enregistrements ? Avec un million d'enregistrements ?)
- Les acteurs sont-ils identifiés pour chaque action ? (Pas "le système valide" mais "le système valide après approbation du gestionnaire de niveau 2")
- Les données d'entrée et de sortie sont-elles spécifiées ? (Quels champs ? Quels formats ? Quelles contraintes ?)

Pour le projet Helios, Nadia et Émilie évaluent la complétude à 2/5. La spécification client décrit bien les fonctionnalités dans leur cas nominal, mais mentionne rarement les cas d'erreur et ne précise pas les critères d'acceptation pour la majorité des exigences.

Dimension 2 — Précision

La dimension de précision évalue si chaque affirmation dans la spécification peut être comprise de façon univoque par toutes les parties.

- Chaque terme clé du domaine est-il défini dans un glossaire ou dans la spec elle-même ?
- Les quantités sont-elles exprimées en valeurs numériques mesurables ? ("moins de 3 secondes" plutôt que "rapide")

- Les conditions sont-elles exprimées de façon binaire (vrai/faux) plutôt que qualitative ? ("si le solde est négatif" plutôt que "si la situation financière est problématique")
- Les acteurs sont-ils identifiés par leur rôle précis plutôt que par des termes génériques ? ("l'administrateur système" plutôt que "les utilisateurs autorisés")
- Les références à d'autres systèmes ou standards sont-elles précises ? ("selon la norme ISO 27001:2022" plutôt que "selon les standards de sécurité")

Pour le projet Helios : précision évaluée à 2/5. Le document utilise 23 termes non définis que l'équipe identifie comme potentiellement ambigus. Les performances sont décrites de façon qualitative dans 80% des cas.

Dimension 3 — Cohérence

La dimension de cohérence évalue si les exigences ne se contredisent pas entre elles et si les priorités sont alignées.

- Deux exigences se contredisent-elles directement ? ("Le système doit traiter toutes les demandes en temps réel" vs "Le système effectue un batch de traitement quotidien")
- Les priorités sont-elles cohérentes entre elles ? (Une fonctionnalité de priorité haute dépend-elle d'une fonctionnalité de priorité basse ?)
- Les exigences fonctionnelles et non-fonctionnelles sont-elles compatibles ? (Exiger une performance de 100ms pour une requête qui nécessite de consulter 5 systèmes différents)
- Les versions différentes du document (s'il y en a plusieurs) sont-elles alignées ?
- Les exigences client sont-elles cohérentes avec les contraintes techniques exprimées par l'équipe ?

Pour le projet Helios : cohérence évaluée à 3/5. Deux contradictions directes sont identifiées, et trois cas de dépendances de priorité problématiques.

Dimension 4 — Testabilité

La dimension de testabilité est le test ultime de clarté d'une spécification. Si une exigence ne peut pas être transformée en test automatisé ou en scénario de test manuel précis, elle n'est pas encore une spécification — c'est une intention.

- Peut-on écrire un test de type GIVEN/WHEN/THEN pour chaque exigence sans avoir à inventer des données ou des conditions non spécifiées ?
- Le critère d'acceptation est-il objectif ? (Peut-on décider de façon non ambiguë si l'exigence est satisfaite ou non ?)
- Les données de test nécessaires pour valider l'exigence peuvent-elles être préparées à partir des informations disponibles dans la spec ?
- Un testeur qui n'a pas participé aux réunions d'élicitation peut-il écrire les cas de test à partir de la seule lecture de la spec ?

Pour le projet Helios : testabilité évaluée à 1.5/5. La grande majorité des exigences ne peut pas être directement transformée en tests sans interprétation significative.

Outil terrain — Grille d'audit de spécification

Dimension

Score 1

Score 3

Score 5

Complétude

Cas nominaux seulement, pas de cas d'erreur, pas de critères d'acceptation

Certains cas d'erreur couverts, quelques critères d'acceptation

Tous les cas couverts (nominal + erreur + limites), critères d'acceptation pour toutes les exigences

Précision

Termes non définis, valeurs qualitatives ("rapide", "simple"), acteurs génériques

Glossaire partiel, certaines valeurs numériques, acteurs parfois précisés

Glossaire complet, toutes les valeurs numériques, tous les acteurs identifiés par leur rôle

Cohérence

Contradictions directes identifiables, priorités non alignées

Quelques tensions mais pas de contradictions directes, priorités globalement cohérentes

Aucune contradiction, priorités pleinement alignées, exigences fonctionnelles et non-fonctionnelles compatibles

Testabilité

Impossible d'écrire des tests sans interprétation majeure

Tests possibles pour certaines exigences avec quelques hypothèses

Tests écrits directement à partir de la spec, sans interprétation

Interprétation du score total (4 à 20) :
4–8 : Spec à risque élevé — session d'élicitation complète requise avant tout développement
9–13 : Spec à risque modéré — raffinement ciblé nécessaire pour les dimensions les plus faibles
14–17 : Spec acceptable — validation et quelques clarifications suffisent
18–20 : Spec de haute qualité — prête pour le développement

Le score du projet Helios est de 8.5/20. L'audit dit clairement : ce projet ne peut pas démarrer en développement sans une phase significative de clarification. Nadia accepte ce verdict. Elle planifie trois semaines de sessions d'élicitation structurée

avec Anne avant le premier sprint de développement. C'est trois semaines qu'elle n'avait pas prévu dans le plan initial. Et c'est trois semaines qui éviteront neuf mois de travail dans la mauvaise direction.

2.2 Les 7 mots dangereux dans une spécification

L'audit de spécification en 4 dimensions donne une vue globale du niveau de clarté d'un document. Mais il y a une façon encore plus rapide d'identifier les zones à risque : chercher les 7 mots dangereux.

Ces mots sont dangereux non pas parce qu'ils sont incorrects, mais parce qu'ils sont systématiquement interprétés différemment selon le cadre de référence du lecteur. Ils créent l'illusion de la précision tout en masquant l'ambiguïté. Chaque fois qu'un de ces mots apparaît dans une spec sans être immédiatement suivi d'une définition précise, c'est une bombe à retardement.

Mot dangereux n°1 — RAPIDE

"Le système doit être rapide." "La recherche doit retourner des résultats rapidement." "Le chargement doit être rapide."

Pour une directrice métier habituée à des systèmes legacy qui prennent 30 secondes à charger, "rapide" signifie peut-être 5 secondes. Pour un développeur formé aux standards web modernes, "rapide" signifie peut-être moins de 500ms. Pour un architecte qui pense aux contraintes de performance sur 5 ans d'utilisation, "rapide" est une conversation entière sur les SLAs et les percentiles de performance.

Transformation requise : Toute mention de "rapide" doit être remplacée par une valeur numérique mesurable, dans des conditions d'utilisation spécifiées et pour un volume de données défini. Exemple : "La recherche doit retourner les 50 premiers résultats en moins de 1,5 secondes pour une base de données de 500 000 enregistrements avec une charge simultanée de 100 utilisateurs actifs."

Mot dangereux n°2 — SIMPLE

"L'interface doit être simple." "Le processus doit être simple à utiliser." "La configuration doit être simple."

La simplicité est intrinsèquement subjective et culturellement relative. Ce qui est simple pour un power-user qui maîtrise les raccourcis clavier peut être complexe pour un utilisateur occasionnel. Ce qui est simple pour un administrateur système peut être opaque pour un utilisateur métier. Et ce qui est simple dans un contexte anglophone peut être complexe dans un contexte multilingue.

Transformation requise : "Simple" doit être remplacé par une description du profil d'utilisateur cible et de la tâche spécifique à accomplir. Exemple : "Un utilisateur ayant une formation de base (2 heures) doit pouvoir réaliser les opérations courantes (création, modification, suppression d'un enregistrement) sans assistance en moins de 5 minutes."

Mot dangereux n°3 — INTUITIF

"L'interface doit être intuitive." "Le workflow doit être intuitif." "La navigation doit être intuitive."

L'intuitivité est encore plus subjective que la simplicité — elle fait référence à des modèles mentaux qui varient considérablement selon l'expérience antérieure de l'utilisateur. Une interface "intuitive" pour quelqu'un qui a grandi avec les smartphones est potentiellement déstabilisante pour quelqu'un dont l'expérience numérique s'est construite sur des terminaux textuels.

Transformation requise : "Intuitif" doit être traduit en critères de test utilisateur mesurables. Exemple : "Un utilisateur nouvellement recruté doit pouvoir compléter le workflow d'onboarding sans assistance externe lors d'un test utilisateur supervisé."

Mot dangereux n°4 — FLEXIBLE

"Le système doit être flexible." "La configuration doit être flexible." "Le reporting doit être flexible."

La flexibilité a un coût architectural direct. Un système "flexible" peut signifier paramétrable par l'administrateur, configurable par l'utilisateur final, ou architecturalement extensible pour des besoins futurs non encore définis. Ces trois définitions impliquent des architectures et des efforts de développement fondamentalement différents.

Transformation requise : Identifier précisément ce qui doit être flexible, pour qui, et dans quelles limites. Exemple : "L'administrateur système peut modifier la liste des statuts disponibles dans le workflow d'approbation sans intervention du développeur, dans la limite de 20 statuts maximum."

Mot dangereux n°5 — SÉCURISÉ

"Le système doit être sécurisé." "Les données doivent être sécurisées." "L'accès doit être sécurisé."

La sécurité est un spectre qui va du mot de passe simple à la conformité SOC 2 Type II, en passant par HTTPS, chiffrement au repos, authentification multi-facteurs, gestion des clés cryptographiques, audit logging, penetration testing, et bien davantage. Sans précision sur le niveau de sécurité requis, l'équipe doit faire une hypothèse — et cette hypothèse peut être soit insuffisante (exposant l'organisation à des risques réels) soit excessive (engendrant des coûts et des délais non prévus).

Transformation requise : Référencer le standard de sécurité applicable et identifier les exigences spécifiques. Exemple : "L'authentification doit utiliser OAuth 2.0 avec PKCE, les sessions expirent après 30 minutes d'inactivité, et toutes les données personnelles sont chiffrées au repos avec AES-256."

Mot dangereux n°6 — APPROPRIÉ

"Le message d'erreur doit être approprié." "Le format de l'export doit être approprié." "Le délai de réponse doit être approprié."

"Approprié" est le mot qui délègue entièrement la définition du comportement attendu au jugement du développeur. C'est une invitation à l'hypothèse non documentée. Et les hypothèses non documentées sont le terreau des malentendus.

Transformation requise : Spécifier ce qu'est le comportement "approprié" dans ce contexte spécifique. Pour un message d'erreur : "Le message d'erreur indique le champ en erreur, la valeur soumise, et la valeur attendue selon le format [exemple], en français uniquement."

Mot dangereux n°7 — ET/OU

"L'utilisateur doit saisir son nom et/ou son prénom." "La validation est réussie si la date et/ou le montant est correct." "La notification est envoyée par email et/ou SMS."

Le "et/ou" est l'ambiguïté grammaticale la plus facilement évitable — et l'une des plus fréquentes. "Et/ou" peut signifier "les deux", "l'un des deux", ou "au moins l'un des deux". Ces trois interprétations produisent trois comportements système différents. En logique formelle, "ou" est inclusif (l'un ou l'autre ou les deux) — mais ce n'est pas nécessairement l'interprétation du lecteur.

Transformation requise : Remplacer systématiquement "et/ou" par une formulation précise. "L'utilisateur doit saisir son nom et son prénom" (les deux obligatoires). Ou "L'utilisateur doit saisir au moins son nom ou son prénom" (un des deux suffit). Ou "L'utilisateur peut saisir son nom, son prénom, ou les deux" (tous optionnels, au moins un requis).

Outil terrain — La chasse aux 7 mots dangereux

Procédure : utiliser la fonction de recherche (Ctrl+F) dans le document de spécification pour trouver chacun des 7 mots dangereux. Pour chaque occurrence :

1. Surligner l'occurrence en jaune.
2. Ajouter un commentaire : "Ambiguïté de type [précision/mesure/définition] — à clarifier avant développement."
3. Loguer dans le registre d'ambiguïtés avec owner et deadline.

Dans le projet Helios, cette procédure révèle 34 occurrences des 7 mots

dangereux dans le document de 47 pages. Chacune est une zone de risque. Chacune est une opportunité d'éliminer un malentendu potentiel.

Mots à rechercher : rapide, simple, intuitif, flexible, sécurisé, approprié, et/ou (et leurs variantes : rapidement, simplement, intuitivement, flexiblement, etc.)

2.3 Le glossaire de projet comme infrastructure de clarté

L'audit a révélé l'étendue des ambiguïtés. La chasse aux 7 mots dangereux a identifié les zones à risque. Maintenant, il faut construire l'infrastructure qui va prévenir la réapparition de ces ambiguïtés tout au long du projet.

Cette infrastructure s'appelle le **glossaire de projet**. Et son principe est simple mais puissant : chaque terme clé du projet reçoit une définition précise, négociée explicitement entre toutes les parties, documentée et accessible à tous, et référencée dans chaque spécification qui utilise ce terme.

Le glossaire de projet est différent d'un dictionnaire général. Un dictionnaire donne la définition générale d'un mot. Le glossaire de projet donne la définition de ce mot *dans le contexte de ce projet*. Et ces deux définitions sont souvent très différentes.

La construction du glossaire — processus en 3 étapes

Étape 1 — L'identification des termes candidats

Émilie commence par identifier tous les termes du domaine qui apparaissent dans la spécification et qui pourraient avoir des significations différentes selon le contexte. Elle utilise deux sources :

Source 1 : les termes qui apparaissent dans la spec et qui ne sont pas définis dans un vocabulaire standard universellement partagé. "Notification", "validation", "rapport", "utilisateur", "workflow", "exception".

Source 2 : les termes qui ont fait l'objet d'hésitations, de questions ou de discussions lors des réunions d'élicitation. Si deux personnes ont utilisé le même mot de façon différente dans une même réunion, ce mot est un candidat prioritaire au glossaire.

Pour le projet Helios, Émilie identifie 31 termes candidats. Elle en priorise 12 comme "critiques" — les termes dont l'ambiguïté pourrait avoir un impact majeur sur l'architecture ou la fonctionnalité.

Étape 2 — La session de définition négociée

Nadia organise une session de deux heures avec Anne, Émilie, Rafael et Jonas. L'objectif est de définir ensemble, terme par terme, ce que chaque mot signifie dans le contexte du projet Helios.

Le processus pour chaque terme :

6. Anne donne sa définition ("pour moi, une notification, c'est...").
7. Jonas donne sa définition ("techniquement, une notification peut être...").
8. Rafael donne sa perspective architecturale ("d'un point de vue système, notification implique...").
9. Émilie propose une définition de synthèse.
10. La définition est discutée jusqu'à l'accord de toutes les parties.
11. La définition est notée dans le glossaire avec la mention "Validé par :" et la liste des parties prenantes présentes.

Pour le terme "notification" dans le projet Helios, la session produit la définition suivante :

Exemple — Définition négociée dans le glossaire Helios

NOTIFICATION (dans le contexte du projet Helios)

Définition : Message automatique envoyé par le système à un ou plusieurs utilisateurs identifiés pour les informer d'un événement nécessitant leur

attention ou leur action.

Canal dans ce projet : Message in-app uniquement (affiché dans le centre de notifications de l'interface). Les emails ne sont PAS des notifications dans le contexte de ce projet — ils sont des "communications automatiques" (voir entrée séparée au glossaire).

Déclencheur : Les notifications sont déclenchées par des événements système (liste à définir dans les specs individuelles de chaque notification). Elles ne sont PAS envoyées sur action directe de l'utilisateur.

Persistance : Les notifications restent visibles dans le centre de notifications jusqu'à lecture explicite par l'utilisateur. Elles ne disparaissent pas automatiquement.

Validé par : Anne (direction métier), Nadia (gestion de projet), Rafael (architecture), Émilie (analyse), Jonas (développement)

Date de validation : Semaine 2 du projet — à réviser si nécessaire lors de la revue mensuelle du glossaire.

Ce niveau de précision peut sembler excessif. Il ne l'est pas. Il est exactement ce qui évite que "notification" soit implémentée de trois façons différentes par trois développeurs différents au cours du projet.

Étape 3 — Le maintien vivant du glossaire

Le glossaire n'est pas un document figé. C'est un artefact vivant qui évolue avec le projet. Nadia établit trois règles de gestion :

Règle 1 — Le glossaire est la première référence. Avant d'utiliser un terme dans une spec, on vérifie s'il est dans le glossaire. S'il l'est, on utilise la définition du glossaire. S'il ne l'est pas, on l'ajoute avant d'écrire la spec.

Règle 2 — Le glossaire est modifiable mais tracé. Si une définition doit évoluer en cours de projet, la modification est signalée à toutes les parties concernées et

validée de la même façon que la définition initiale. Les specs qui utilisaient l'ancienne définition sont revues pour s'assurer qu'elles sont toujours cohérentes.

Règle 3 — Le glossaire est revenu au client. Chaque définition est soumise à la validation d'Anne. Le glossaire n'est pas un document interne de l'équipe — c'est un accord contractuel avec le client sur le sens des mots. Anne le signe. Et cette signature a une valeur : si lors d'une démo Anne dit "ce n'est pas ce que je voulais dire", on ouvre le glossaire. Si le terme est défini et validé par elle, la conversation est différente.

Ce qui n'a pas fonctionné — Le glossaire imposé

Dans le premier projet où Émilie avait tenté d'introduire un glossaire, elle l'avait construit seule, en interne, et l'avait distribué à l'équipe comme référence. Le glossaire avait été ignoré. Personne ne l'avait consulté. Les définitions qu'il contenait reflétaient la compréhension d'Émilie — pas la compréhension négociée de toutes les parties.

La leçon : un glossaire construit sans le client n'est pas un accord. C'est un monologue. La valeur du glossaire vient du processus de négociation, pas du document résultant. Le processus force la conversation. Le document formalise l'accord.

2.4 La session d'élicitation structurée — outil terrain

L'audit a évalué la qualité de la spécification existante. La chasse aux 7 mots dangereux a identifié les zones à risque. Le glossaire a défini les termes clés. Maintenant, il faut combler les lacunes — obtenir d'Anne les informations qui manquent pour avoir une spécification complète.

C'est le rôle de la session d'élicitation structurée. Et "structurée" est le mot-clé : une réunion non structurée avec un client pour "clarifier les besoins" produit généralement une heure de conversation qui revient au point de départ. Une session structurée produit des données exploitables.

Salle de réunion client — Semaine 3 du projet Helios — Émilie anime

Anne est assise en face d'Émilie. C'est la première fois qu'Émilie anime une réunion avec Anne directement — sans Nadia pour "gérer" la relation client. Nadia a décidé de ne pas venir : "C'est ta réunion. Tu sais ce qu'on cherche."

Émilie a son carnet. Et une structure.

Elle commence : « Anne, merci d'être là. L'objectif de cette session est de comprendre les fonctionnalités de la section 4 de votre document avec beaucoup plus de précision. Je vais vous poser des questions dans un format spécifique — parfois les questions sembleront évidentes, mais les réponses que vous me donnez sont exactement ce dont nous avons besoin pour construire ce qui vous servira vraiment. Ça vous convient ? »

Anne : « Bien sûr. »

Émilie : « Première fonctionnalité : la gestion des alertes de conformité. Commençons par le contexte. Pourquoi ce besoin existe-t-il aujourd'hui ? Qu'est-ce que vous ne pouvez pas faire avec vos systèmes actuels que vous voulez pouvoir faire avec ce nouveau système ? »

Anne prend une grande inspiration. Et commence à parler. Pendant 12 minutes, sans interruption de la part d'Émilie. Elle parle du processus actuel, manuel et chronophage, des tableaux Excel partagés par email, des délais de réaction quand un seuil est dépassé, des audits qui révèlent des lacunes dans la traçabilité...

Émilie note. Elle ne reformule pas encore. Elle écoute.

Quand Anne s'arrête, Émilie : « Merci. Maintenant le scénario nominal : décrivez-moi, étape par étape, comment ce processus fonctionnerait dans le

système idéal pour vous. Du moment où un indicateur dépasse le seuil jusqu'au moment où la situation est résolue. »

La session d'élicitation structurée qu'Émilie utilise est organisée en **quatre temps**, appliqués systématiquement pour chaque fonctionnalité ou groupe de fonctionnalités à clarifier :

Temps 1 — Le contexte (10-15 minutes par fonctionnalité)

La question fondamentale : *Pourquoi ce besoin existe-t-il ?*

Le contexte est la porte d'entrée sur l'intention réelle derrière une demande. "Pourquoi ce besoin existe" révèle souvent que le besoin exprimé est une solution à un problème sous-jacent — et que d'autres solutions existent parfois. Comprendre le contexte permet de valider que la fonctionnalité demandée est bien la réponse appropriée au problème réel.

Questions de contexte :

- Qu'est-ce que vous faites aujourd'hui pour gérer ce besoin, et pourquoi ça ne suffit plus ?
- Qu'est-ce que vous voulez être capable de faire demain que vous ne pouvez pas faire aujourd'hui ?
- Quel problème concret cette fonctionnalité résout-elle ? Pouvez-vous donner un exemple récent de ce problème ?
- Qui est directement impacté quand ce problème se produit ?

Temps 2 — Le scénario nominal (15-20 minutes par fonctionnalité)

La question fondamentale : *Décrivez le cas qui marche — étape par étape.*

Le scénario nominal est la description du comportement attendu dans les conditions idéales. Cette description doit être suffisamment détaillée pour qu'un développeur

puisse l'implémenter sans poser d'autre question. Les bons scénarios nominaux identifient les acteurs à chaque étape, les actions, les données impliquées, et les résultats attendus.

Questions de scénario nominal :

- Qui déclenche cette fonctionnalité ? Que fait-il/elle exactement pour la déclencher ?
- Qu'est-ce qui se passe ensuite ? Et ensuite ? (Détailler chaque étape jusqu'à la fin du processus)
- Qu'est-ce qui change dans le système après que cette fonctionnalité a été exécutée avec succès ?
- Comment l'utilisateur sait-il que c'est terminé et réussi ?

Temps 3 — Les cas d'exception (15-20 minutes par fonctionnalité)

La question fondamentale : *Que se passe-t-il si ça ne marche pas comme prévu ?*

C'est souvent le temps le plus révélateur de la session. La majorité des clients n'ont jamais eu à réfléchir systématiquement aux cas d'exception — parce que dans leur processus actuel, les exceptions sont gérées de façon informelle, souvent par la même personne qui a de l'expérience. Quand on leur pose la question, leur première réponse est souvent "ça ne devrait pas arriver". La deuxième réponse, après quelques secondes de réflexion, est "mais si ça arrive...".

Questions de cas d'exception :

- Que se passe-t-il si les données sont incomplètes ou incorrectes ?
- Que se passe-t-il si l'utilisateur change d'avis à mi-parcours ?
- Que se passe-t-il si le délai est dépassé ?
- Que se passe-t-il si le système de référence est indisponible ?
- Que se passe-t-il si l'utilisateur qui doit approuver est absent ?
- Y a-t-il des règles métier particulières pour certaines catégories de données ?

Temps 4 — La définition du succès (10 minutes par fonctionnalité)

La question fondamentale : *Comment saurez-vous que c'est réussi ?*

Cette question est la plus directement liée à la testabilité de la spécification. Si Anne ne peut pas répondre à cette question de façon précise — si elle ne peut pas dire "je saurai que c'est réussi quand..." — alors la fonctionnalité n'est pas encore suffisamment définie pour être développée.

Questions de définition du succès :

- Dans 6 mois, qu'est-ce qui aura changé dans votre façon de travailler grâce à cette fonctionnalité ?
- Si vous deviez tester cette fonctionnalité vous-même, que feriez-vous pour vérifier qu'elle fonctionne correctement ?
- Y a-t-il un résultat chiffré que vous attendez ? (Réduction du temps de traitement, élimination d'une certaine catégorie d'erreurs, etc.)
- Qui, dans votre organisation, pourra valider lors de la démo que la fonctionnalité est conforme à vos attentes ?

Suite de la session — Deux heures plus tard

Émilie ferme son carnet. Elle a rempli 18 pages de notes. Elle a couvert 3 des 7 fonctionnalités de la section 4.

Anne : « Je n'aurais jamais su expliquer ce que je voulais aussi précisément sans vos questions. »

Émilie : « Vos réponses aux questions sur les cas d'exception ont été particulièrement utiles. »

Anne : « Je ne m'étais jamais posé ces questions. Je pensais que c'était "évident" comment ça devait fonctionner quand quelque chose se passe mal.

Mais en y réfléchissant... il y a des règles assez complexes. »

Émilie note mentalement : 8 exigences implicites qu'Anne considérait "évidentes" — et qui auraient normalement été découvertes lors d'une démo de sprint, après 3 semaines de développement dans la mauvaise direction. Cette session de 2 heures vient d'éviter 3 semaines de refonte.

Le retour sur investissement d'une session d'élicitation structurée est considérable. Deux heures d'élicitation évitent typiquement entre 2 et 6 semaines de refonte. Le calcul est simple — et il devrait être fait explicitement lors de la planification des projets pour justifier l'investissement en temps.

2.5 Le test du taxi

Jonas avait un problème récurrent. Les specs qu'il recevait avant chaque sprint étaient techniquement en bon ordre — correctement structurées, avec des critères d'acceptation — mais quand il commençait à coder, il découvrait invariablement des zones grises. Des situations que la spec ne couvrait pas clairement. Des cas que personne n'avait pensé à spécifier.

Il avait développé sa propre pratique pour tester la clarté d'une spec avant de commencer à coder. Il l'appelait "le test du taxi" — et il l'expliquait ainsi :

Jonas — Présentation du test du taxi à l'équipe

« Le test du taxi, c'est simple : avant de commencer une story, j'imagine que je dois l'expliquer à mon chauffeur de taxi. Pas à un développeur. Pas à quelqu'un qui connaît le contexte du projet. À quelqu'un qui n'a aucune connaissance préalable. Et j'essaie de l'expliquer en deux minutes de façon à ce qu'il comprenne exactement ce que le système doit faire — pas en termes

techniques, mais en termes de comportement observable. »

Émilie : « Et si tu ne peux pas ? »

Jonas : « Si je ne peux pas expliquer clairement la story à un non-spécialiste en 2 minutes, c'est que je ne la comprends pas moi-même suffisamment clairement pour la coder. Il me manque quelque chose. Soit je retourne la story à Émilie pour clarification, soit je note mes questions spécifiques et je les pose avant de commencer. »

Rafael : « Et ça marche ? »

Jonas : « Depuis que je le fais systématiquement, j'ai réduit de moitié le nombre de fois où je reviens en cours de sprint avec des questions de clarification. Et j'ai éliminé presque complètement les cas où je livre quelque chose que personne ne voulait vraiment. »

Le test du taxi est un outil de vérification individuelle — une pratique que Jonas applique avant de commencer chaque story. Il ne remplace pas les outils collectifs (l'audit de spec, le glossaire, la session d'élicitation). Il les complète en ajoutant une couche de vérification personnelle au niveau de l'implémentation.

La puissance du test du taxi tient dans ce qu'il révèle : si un développeur expérimenté ne peut pas expliquer clairement une fonctionnalité à un non-spécialiste, c'est que sa compréhension de la fonctionnalité contient des hypothèses implicites qu'il n'a pas encore explicitement validées. Ces hypothèses sont les zones grises — et les zones grises sont là où les malentendus vivent.

Variante — Le test du taxi en team

Émilie a proposé une variante collective du test du taxi lors du sprint planning : chaque story est expliquée par son développeur assigné en 2 minutes maximum — sans jargon technique — devant toute l'équipe. L'équipe écoute et signale immédiatement les points qui n'ont pas été couverts ou qui semblent ambigus.

Cette variante a deux avantages supplémentaires :

12. Elle crée un moment formel où les hypothèses implicites du développeur sont exposées à d'autres points de vue — et les ambiguïtés visibles pour certains membres de l'équipe peuvent être révélées.
13. Elle donne à toute l'équipe une compréhension partagée de ce qui est en cours de développement — ce qui facilite la collaboration et le support mutuel pendant le sprint.

Outil terrain — Protocole du test du taxi

Conditions du test :

1. Lire la story complète (description + critères d'acceptation).
2. Fermer le document. Ne pas regarder les notes.
3. Expliquer oralement (ou par écrit) ce que le système fait dans cette story — en termes de comportement observable pour un utilisateur — en 2 minutes maximum.
4. Inclure dans l'explication : qui déclenche la fonctionnalité, ce qui se passe, et comment l'utilisateur sait que c'est terminé.

Interprétation :

- Si l'explication est fluide et complète → la story est prête pour le développement.
- Si l'explication hésite ou s'arrête → noter les points d'hésitation comme questions à clarifier avant de commencer.
- Si l'explication nécessite plus de 2 minutes → la story est trop large ou trop complexe pour être développée en un sprint. Envisager la division.
- Si l'explication contient des hypothèses ("je suppose que...", "normalement ça devrait...") → ces hypothèses doivent être validées avec l'analyste avant le développement.

2.6 Anecdote — Rafael et la « réconciliation des données »

Rafael — Rétrospective d'un projet précédent

Rafael avait 11 ans d'expérience. Il avait vu beaucoup de projets, de bonnes specs et de mauvaises specs, de clients clairs et de clients confus. Mais le cas de "la réconciliation des données" restait pour lui l'exemple le plus parfait d'un terme non défini qui avait détruit trois mois de travail d'architecture.

C'était un projet d'intégration entre deux systèmes financiers hérités. La spec mentionnait, dès la page 2 : "Le système doit assurer la réconciliation des données entre les deux plateformes."

Rafael avait lu cette phrase et avait hoché la tête. Il savait ce qu'était la réconciliation des données. C'était un concept bien connu en finance et en informatique. Il avait conçu l'architecture en conséquence.

Ce qu'il ne savait pas : le terme "réconciliation" avait trois significations différentes pour les trois parties impliquées dans le projet.

Pour l'équipe projet (Rafael inclus), "réconciliation des données" signifiait le processus technique de comparaison et de synchronisation des données entre deux bases de données — identifier les différences, résoudre les conflits selon des règles de priorité, et s'assurer que les deux systèmes sont en état cohérent après synchronisation. Rafael avait conçu une architecture de réconciliation bi-directionnelle avec des règles de résolution de conflits sophistiquées.

Pour le client (la directrice des systèmes d'information), "réconciliation des données" signifiait la production d'un rapport mensuel de rapprochement — un document Excel montrant les écarts entre les deux systèmes, pour revue manuelle par une équipe de comptables. Elle n'avait jamais imaginé une synchronisation

automatique — dans son univers, la réconciliation était un processus manuel de vérification, pas de correction.

Pour l'équipe de comptables (les utilisateurs finaux), "réconciliation des données" signifiait la possibilité de corriger manuellement les écarts dans le système — un workflow de validation et de correction d'entrées, one by one, avec une piste d'audit complète.

Trois définitions. Trois architectures. Trois mois de travail pour la première, abandonnée quand la deuxième définition a émergé. Deux mois supplémentaires pour la deuxième, partiellement reconstruite quand la troisième définition a été comprise. Au total : cinq mois de refonte.

Rafael — Réflexion a posteriori

« Ce que j'ai appris de ce projet : chaque fois que je crois connaître la définition d'un terme, je dois quand même demander. Surtout quand je crois la connaître. Parce que ma connaissance est ma connaissance — pas nécessairement celle du client. »

« Le pire dans cette histoire : j'avais implémenté "réconciliation des données" de façon légèrement différente à trois occasions différentes au cours du projet — parce que chaque fois qu'on avait une nouvelle réunion, j'avais une nouvelle nuance de compréhension. Résultat : trois implémentations partielles, aucune complète. »

« Un terme non défini dans le glossaire est une spécification qui sera implémentée différemment à chaque fois. »

Il s'arrête. « Cette phrase, je me la répète à chaque nouveau projet avant la première réunion d'élicitation. »

L'anecdote de Rafael illustre une dimension souvent sous-estimée de l'ambiguïté lexicale : un terme ambigu n'est pas seulement interprété différemment par

différentes personnes. Il peut être interprété différemment par la même personne à des moments différents, selon le contexte de la conversation du jour. C'est l'instabilité de sens — et elle est aussi dangereuse que l'ambiguïté initiale.

Un glossaire de projet protège contre l'instabilité de sens autant que contre l'ambiguïté interpersonnelle. La définition écrite et validée est une ancre. Elle fixe le sens dans le temps. Et quand une nouvelle réunion apporte une nouvelle nuance, la question devient : "Est-ce une nuance compatible avec notre définition dans le glossaire, ou est-ce que notre définition doit être mise à jour ?" — et non pas "Comment est-ce que j'interprète ce que j'ai entendu aujourd'hui ?"

2.7 Scène DREAM-TEAM — Théo au tableau blanc

ORIGINALIUM — Salle de sprint planning — Mardi matin

Théo est debout devant le tableau blanc. L'équipe est assise en arc de cercle : Rania avec son ordinateur, Karim avec son carnet de test, deux développeurs juniors qui viennent de rejoindre l'équipe.

Sur le tableau : une liste de 15 questions. Théo l'appelle "La liste anti-ambiguïtés". Il la sort à chaque sprint planning, pour chaque story qui n'est pas encore passée par le processus complet d'élicitation.

Théo : « Pour chaque story qu'on met dans le sprint, on répond à ces 15 questions. Si on bloque sur une, la story attend. »

Rania lève la main : « La question que je préfère : numéro 9. "Que se passe-t-il si l'utilisateur fait exactement le contraire de ce qu'on attend ?" Cette question révèle des cas d'exception que personne n'avait imaginés. »

Karim : « Moi c'est numéro 1 : "Comment saurons-nous que c'est terminé ?" Si on ne peut pas répondre en 30 secondes avec des critères observables, la

story n'est pas prête. »

Un développeur junior : « Et la numéro 12 : "Ce comportement est-il le même pour tous les types d'utilisateurs ?" — j'aurais mis du temps à penser à poser cette question avant qu'on l'ait sur la liste. »

Théo : « C'est le point. Toutes ces questions, quelqu'un dans l'équipe les aurait eues à un moment. Mais peut-être trop tard. La liste les sort en sprint planning, quand il est encore temps de faire quelque chose. »

La liste anti-ambiguïtés d'ORIGINALIUM est présentée ici dans sa version complète :

#	Question	Type d'ambiguïté adressée
1	Comment saurons-nous que c'est terminé ? (critères observables en 30 secondes)	Testabilité
2	Qui exactement déclenche cette fonctionnalité ?	Structurelle
3	Tous les termes clés de cette story sont-ils dans le glossaire ?	Lexicale
4	Qu'est-ce qui est explicitement exclu du périmètre de cette story ?	De portée
5	Dans quel délai précis les actions du système doivent-elles se produire ?	Temporelle
6	Que se passe-t-il si les données d'entrée sont manquantes ou incorrectes ?	Conditionnelle
7	Que se passe-t-il si la connexion réseau est interrompue en cours d'action ?	Conditionnelle

#	Question	Type d'ambiguïté adressée
8	Que se passe-t-il si l'utilisateur annule à mi-parcours ?	Conditionnelle
9	Que se passe-t-il si l'utilisateur fait exactement le contraire de ce qu'on attend ?	Conditionnelle
10	Y a-t-il des règles métier implicites qui ne sont pas écrites dans la spec ?	Complétude
11	Cette story dépend-elle d'une autre story non encore développée ?	Cohérence
12	Ce comportement est-il le même pour tous les types d'utilisateurs ?	Structurelle
13	La spec contient-elle un des 7 mots dangereux non encore remplacé ?	Précision
14	Le client a-t-il validé les critères d'acceptation de cette story ?	Cohérence
15	Un testeur peut-il écrire les cas de test de cette story sans poser de question supplémentaire ?	Testabilité

La liste anti-ambiguïtés n'est pas une procédure bureaucratique. C'est un outil cognitif. Elle représente les questions que les meilleurs membres de l'équipe posent instinctivement — et elle les rend accessibles à toute l'équipe, y compris les membres moins expérimentés.

Théo explique comment utiliser la liste efficacement : "On ne répond pas à toutes les questions pour toutes les stories. Certaines questions sont clairement hors-sujet selon le type de story. Mais le simple fait de les lire déclenche une réflexion qui souvent révèle une ambiguïté qu'on n'avait pas vue. La liste est un déclencheur de pensée, pas une checklist à cocher mécaniquement."

📌 À retenir — Chapitre 2

- L'**audit de spécification en 4 dimensions** (complétude, précision, cohérence, testabilité) donne un score de maturité qui oriente les priorités de clarification. Un score inférieur à 9/20 nécessite une phase d'élicitation complète avant tout développement.
- Les **7 mots dangereux** (rapide, simple, intuitif, flexible, sécurisé, approprié, et/ou) sont des signaux d'ambiguïté systématique. Chaque occurrence non accompagnée d'une définition précise est une bombe à retardement.
- Le **glossaire de projet** est un accord négocié, pas un document interne. Sa valeur vient du processus de négociation avec le client, pas du document résultant. Chaque terme défini dans le glossaire est une ambiguïté éliminée.
- La **session d'élicitation structurée en 4 temps** (contexte, scénario nominal, cas d'exception, définition du succès) révèle systématiquement les exigences implicites que le client considère "évidentes" — et qui sont invisibles dans les specs initiales.
- Le **test du taxi** est une vérification individuelle : si un développeur ne peut pas expliquer une story à un non-spécialiste en 2 minutes, la story contient des hypothèses implicites non validées.
- Un terme non défini dans le glossaire sera implémenté **différemment à chaque itération** — par le même développeur, selon le contexte de la dernière réunion. C'est l'instabilité de sens, aussi dangereuse que l'ambiguïté initiale.

🔍 Symptômes à observer dans votre équipe

- Les specs commencent le développement sans avoir été auditées — on "avance" sur la base de la compréhension initiale.
- Les termes clés du projet ne sont définis nulle part — chacun travaille avec sa propre définition implicite.
- Les sessions de clarification avec le client sont non structurées — elles produisent des notes mais rarement des décisions exploitables.
- Les développeurs posent des questions de clarification en cours de sprint — signal que la spec n'était pas suffisamment précise avant le début du développement.
- Les specs contiennent des mots comme "rapide", "simple", "intuitif" sans mesures associées.
- Le même terme est utilisé différemment dans différentes parties de la spec — sans que personne ne l'ait remarqué.

💬 **Questions de réflexion — Chapitre 2**

1. Prenez une spec récente de votre équipe et appliquez l'audit en 4 dimensions. Quel est le score ? Quelle dimension est la plus faible ? Qu'est-ce que cela implique ?
2. Faites la chasse aux 7 mots dangereux dans une de vos specs récentes. Combien d'occurrences trouvez-vous ? Qu'aurait-il fallu écrire à la place ?
3. Votre équipe a-t-elle un glossaire de projet ? S'il existe, est-il utilisé ? S'il n'existe pas, quels termes clés de votre projet actuel gagneraient à être définis ?
4. Appliquiez-vous déjà une structure aux sessions d'élicitation avec vos

clients ? Si oui, incluait-elle systématiquement les cas d'exception ? Si non, qu'est-ce que vous pensez qu'une structure ajouterait ?

5. Imaginez appliquer le test du taxi à votre dernière story développée. Auriez-vous pu l'expliquer clairement en 2 minutes avant de commencer ? Après ?

⚠️ **Signal de vigilance — Chapitre 2**

Le test de testabilité : prenez la spécification de votre prochain sprint. Demandez à votre QA (ou à la personne qui aura à tester) d'écrire les cas de test AVANT que le développement commence — sur la base de la seule lecture de la spec. Si votre QA ne peut pas le faire sans venir poser des questions à l'analyste ou au développeur, votre spec n'est pas prête. Et si votre équipe n'a pas de QA, demandez au développeur junior de faire l'exercice : si lui ne peut pas écrire les tests, le développeur senior qui va implémenter travaille sur des hypothèses non validées.

CHAPITRE 3

La spécification IF…THEN… : le format qui élimine les ambiguïtés

> *« Une spécification sans condition est une invitation. Une spécification IF…*

> *THEN… est un contrat. »*

Les outils des chapitres précédents permettent de diagnostiquer et de prévenir les ambiguïtés. Le format IF…THEN… est l'outil qui élimine les ambiguïtés en les rendant structurellement impossibles. Il ne prévient pas les mauvaises spécifications — il rend les mauvaises spécifications visibles immédiatement, avant que quiconque commence à coder.

Le format IF…THEN… est présenté en détail dans L3 de la collection. Dans le contexte du Guide Jaune, il est appliqué comme outil de terrain dans un projet réel, avec des exemples concrets de transformations de spécifications floues en spécifications précises. L'objectif de ce chapitre n'est pas de présenter le format — c'est de montrer comment l'utiliser, quand l'utiliser, et pourquoi il change fondamentalement la relation de l'équipe avec la clarté.

3.1 Rappel du format IF…THEN… (référence L3)

Le format IF…THEN… est une structure de spécification comportementale. Son principe est simple : chaque comportement du système est exprimé comme une relation causale entre une condition (IF) et un résultat (THEN). Cette structure force l'auteur de la spec à expliciter :

- **Le contexte (IF)** : dans quelles conditions ce comportement se produit-il ? Qui est impliqué ? Quelle est l'état du système avant l'action ?
- **Le comportement (THEN)** : que fait exactement le système ? Quelle est la réponse précise et observable ?
- **Et implicitement** : les acteurs (qui fait l'action dans le IF ?), les données (quelles données sont impliquées ?), les contraintes (dans quelles limites ?)

La puissance du format vient de ce qu'il révèle par son absence. Quand une spec ne peut pas être formulée en IF…THEN… sans hypothèses ou hésitations, c'est la preuve qu'elle contient des ambiguïtés non résolues. Le format est un révélateur d'ambiguïté autant qu'un outil de précision.

La structure complète du format IF…THEN… dans le SDLC réconcilié

Dans le SDLC réconcilié, le format IF…THEN… s'organise en trois couches :

Couche 1 — Le cas nominal

IF [condition : état du système ou action de l'acteur] THEN [résultat attendu] [dans un délai ou à une fréquence précise].

Couche 2 — Les cas d'exception

IF [même condition ou condition proche] AND [condition d'exception] THEN [résultat d'exception] [avec les informations données à l'utilisateur].

Couche 3 — Les conditions limites

IF [condition à la limite des valeurs acceptables] THEN [comportement précis] [avec indication si la limite est inclusive ou exclusive].

Une spécification complète couvre les trois couches pour chaque fonctionnalité. La plupart des spécifications floues ne couvrent que la première — et encore, de façon imprécise.

3.2 La transformation d'une spec floue en IF… THEN… précis

Émilie anime un atelier de transformation avec l'équipe. Elle a préparé 10 spécifications floues typiques — extraites directement du document de spécification du projet Helios — et elle va les transformer en IF…THEN… précis en direct, avec l'équipe.

L'exercice révèle deux choses : premièrement, à quel point les specs floues sont courantes (l'équipe reconnaît chaque exemple comme "typique" de ce qu'elle reçoit). Deuxièmement, à quel point la transformation force des conversations qui n'avaient pas eu lieu.

Transformation 1 — La confirmation

Spec floue : "Le système doit envoyer une confirmation."

Questions révélées par la transformation :

- Confirmation de quoi ? (La soumission du formulaire ? La validation du paiement ? L'approbation par le superviseur ?)
- Confirmation à qui ? (À l'utilisateur qui a soumis ? À d'autres parties prenantes ? À tous les administrateurs ?)
- Par quel canal ? (Email ? In-app ? SMS ?)
- Dans quel délai ?
- Avec quel contenu minimum ?

IF…THEN… précis (après session d'élicitation avec Anne) :

Exemple de transformation complète

Cas nominal :
IF un utilisateur authentifié soumet un formulaire de commande valide (tous les champs obligatoires remplis, montant dans les limites autorisées de son profil) THEN le système envoie automatiquement un email de confirmation à l'adresse email enregistrée dans le profil de l'utilisateur, dans les 5 minutes suivant la soumission, contenant le numéro de référence de la commande, le résumé des items commandés, et la date de traitement prévue.

Cas d'exception 1 (formulaire invalide) :
IF un utilisateur authentifié soumet un formulaire de commande incomplet ou invalide THEN le système affiche immédiatement la liste des champs en erreur avec un message explicatif pour chaque champ, aucun email n'est envoyé, et

l'utilisateur reste sur le formulaire avec les données saisies conservées.

Cas d'exception 2 (montant hors limite) :
IF un utilisateur authentifié soumet un formulaire de commande dont le montant dépasse sa limite d'autorisation THEN le système affiche un message indiquant que la commande nécessite une approbation de niveau supérieur, crée une demande d'approbation dans le système, notifie le superviseur désigné via le centre de notifications, et envoie un email de confirmation à l'utilisateur indiquant que la commande est en attente d'approbation.

Cas d'exception 3 (délai de livraison email) :
IF l'email de confirmation n'a pas été envoyé dans les 5 minutes suivant la soumission en raison d'une indisponibilité du service email THEN le système effectue 3 tentatives supplémentaires à intervalles de 10 minutes. Si les 3 tentatives échouent, le système log l'événement comme erreur critique et notifie l'administrateur système.

La spec floue tenait en 8 mots. La spec précise tient en trois paragraphes — et couvre trois comportements distincts que la version floue laissait entièrement à l'interprétation du développeur.

Transformation 2 — Les préférences utilisateur

Spec floue : "Les utilisateurs peuvent modifier leurs préférences."

IF…THEN… précis :

Exemple de transformation — Préférences utilisateur

Cas nominal :
IF un utilisateur authentifié avec le rôle "Utilisateur Standard" ou "Gestionnaire" accède à la page Paramètres de son compte, section Préférences THEN il peut modifier les champs suivants : langue d'interface (liste de valeurs : FR, EN, ES), fuseau horaire (liste de zones), format de date préféré

(JJ/MM/AAAA ou MM/JJ/AAAA), et fréquence des notifications hebdomadaires (quotidien, hebdomadaire, jamais). Les modifications sont enregistrées immédiatement à la saisie (sans bouton "Enregistrer" distinct) et prennent effet à la prochaine action de l'utilisateur dans l'interface.

Restriction :
IF un utilisateur authentifié avec le rôle "Lecteur" accède à la page Paramètres THEN la section Préférences est visible mais tous les champs sont en mode lecture seule. Un message indique : "Contactez votre administrateur pour modifier vos préférences."

Exclusion explicite :
La page Préférences NE PERMET PAS de modifier : l'adresse email de l'utilisateur (modifiable uniquement par l'administrateur), le mot de passe (dans une section distincte), les permissions et droits d'accès (modifiables uniquement par l'administrateur).

Transformation 3 — La performance

Spec floue : "Le rapport doit être rapide."

IF…THEN… précis :

Exemple de transformation — Performance de rapport

Cas nominal — rapport standard :
IF un utilisateur génère un rapport de type "Synthèse mensuelle" couvrant une période de 1 à 12 mois sur un jeu de données allant jusqu'à 100 000 enregistrements THEN le rapport s'affiche entièrement à l'écran en moins de 3 secondes (temps mesuré depuis le clic sur "Générer" jusqu'à l'affichage complet de la page, dans des conditions de charge normale — définie comme 50 utilisateurs actifs simultanément).

Cas nominal — rapport étendu :

IF un utilisateur génère un rapport de type "Détail complet" couvrant plus de 12 mois ou plus de 100 000 enregistrements THEN le système affiche immédiatement un indicateur de progression, génère le rapport en mode asynchrone, et notifie l'utilisateur via le centre de notifications quand le rapport est disponible. Le délai maximum acceptable pour ce type de rapport est de 5 minutes.

Cas limite — surcharge :
IF plus de 200 utilisateurs sont simultanément actifs et qu'un rapport de type "Détail complet" est lancé THEN le système place la demande en file d'attente, notifie l'utilisateur avec l'estimation du délai, et traite les demandes par ordre d'arrivée.

Transformation 4 — Le workflow d'approbation

Spec floue : "Le système doit gérer les approbations."

IF…THEN… précis :

Exemple de transformation — Workflow d'approbation

Cas nominal :
IF un utilisateur avec le rôle "Initiateur" crée et soumet une demande d'approbation de type [liste définie dans le glossaire] THEN le système :

1. Assigne la demande au premier approbateur selon la règle de routage définie dans la configuration du workflow (voir documentation du module de configuration).
2. Notifie l'approbateur assigné via le centre de notifications in-app ET par email, avec le lien direct vers la demande.
3. Change le statut de la demande à "En attente d'approbation Niveau 1".

4. Enregistre l'horodatage de soumission dans le journal d'audit.

Cas d'exception — délai dépassé :
IF une demande reste en statut "En attente d'approbation" pendant plus de 48 heures ouvrables sans action de l'approbateur THEN le système envoie une notification de rappel à l'approbateur et notifie le gestionnaire de l'approbateur. Si aucune action n'est prise dans les 24 heures supplémentaires, le système escalade au responsable de niveau supérieur défini dans la configuration du workflow.

Cas d'exception — approbateur absent :
IF l'approbateur assigné est en statut "Absent" dans le système (délégation configurée par l'administrateur) THEN la demande est automatiquement redirigée vers le délégué désigné. Si aucun délégué n'est configuré, la demande reste en attente et une alerte est envoyée à l'administrateur.

Observations sur les transformations

En appliquant ces transformations à l'ensemble des fonctionnalités du projet Helios, Émilie documente plusieurs observations importantes :

Observation 1 : Chaque spec floue transformée révèle en moyenne 3 à 5 décisions implicites qui n'avaient pas été explicitées. Ces décisions auraient été prises par le développeur — seul, sans validation client — au moment de l'implémentation.

Observation 2 : La formulation des cas d'exception nécessite systématiquement une session complémentaire avec Anne. Les cas d'exception sont rarement dans la tête du client de façon structurée — ils doivent être "extraits" par des questions spécifiques.

Observation 3 : Certaines specs floues ne peuvent pas être transformées en IF… THEN… parce que les informations nécessaires n'existent pas encore. Elles révèlent des décisions de design qui n'ont pas encore été prises par le client ou par l'équipe. Ces specs doivent être marquées "en attente de décision" et ne peuvent pas entrer en développement avant que la décision soit prise.

Observation 4 : Le temps nécessaire pour transformer une spec floue en IF… THEN… précis est entre 20 minutes et 2 heures selon la complexité. Le temps qui serait nécessaire pour corriger une implémentation basée sur la spec floue serait entre 2 jours et 3 semaines. Le ratio est systématiquement favorable à la transformation préventive.

3.3 Les cas d'exception comme specs à part entière

La transformation des specs floues en IF…THEN… révèle une réalité qui dérange parfois les équipes : les cas d'exception représentent souvent plus de spécification que le cas nominal. Et cette réalité est cohérente avec l'utilisation réelle des systèmes logiciels.

Les utilisateurs d'un système logiciel professionnel ne passent pas leur journée dans le cas nominal. Ils affrontent régulièrement des situations où les données sont incomplètes, où le réseau est lent, où une validation échoue, où un processus est interrompu. Ces situations — les cas d'exception — sont celles où le logiciel peut faire la différence entre un utilisateur efficace et un utilisateur frustré.

Un système qui gère parfaitement le cas nominal mais ignore les cas d'exception n'est pas un système professionnel. C'est un prototype qui attend ses conditions idéales.

La matrice des cas d'exception

Émilie utilise une matrice systématique pour identifier les cas d'exception pour chaque fonctionnalité :

Catégorie d'exception	Questions à poser	Exemples dans le projet Helios
Données invalides	Que se passe-t-il si les données sont manquantes ? Incorrectes ? Hors format ?	Formulaire soumis avec un champ date invalide

Catégorie d'exception	Questions à poser	Exemples dans le projet Helios
Autorisations insuffisantes	Que se passe-t-il si l'utilisateur n'a pas les droits pour cette action ?	Utilisateur "Lecteur" tentant d'approuver une demande
État système invalide	Que se passe-t-il si l'action est demandée sur un objet dans le mauvais état ?	Tentative d'approbation d'une demande déjà annulée
Indisponibilité technique	Que se passe-t-il si un service tiers est indisponible ? La base de données ?	Service d'envoi d'email indisponible lors d'une confirmation
Dépassement de limites	Que se passe-t-il si une valeur dépasse un seuil configuré ?	Commande dépassant la limite d'autorisation de l'utilisateur
Concurrence	Que se passe-t-il si deux utilisateurs modifient le même enregistrement simultanément ?	Deux approbateurs tentant d'approuver la même demande
Interruption de processus	Que se passe-t-il si l'utilisateur ferme l'onglet ? Si la session expire ?	Formulaire à moitié rempli quand la session expire
Volume extrême	Que se passe-t-il avec zéro enregistrements ? Avec un million ?	Rapport généré sur une période sans données

Pour chaque fonctionnalité du projet Helios, Émilie passe en revue les 8 catégories et identifie les cas d'exception applicables. Elle en documente 47 au total — pour 12 fonctionnalités principales. En moyenne, 4 cas d'exception par fonctionnalité. Et chacun de ces 47 cas d'exception est un comportement système qui aurait été "inventé" par le développeur sans cette démarche.

La règle des 80/20 des cas d'exception

Jonas formule une règle pratique issue de son expérience :

> **Jonas — Stand-up — Semaine 5 du projet Helios**

« Il y a une règle que j'ai apprise à la dure dans mes premières années : 80% du temps de développement se passe dans les 20% de cas qui ne sont pas le cas nominal. Le cas nominal, c'est souvent 3 jours de travail. Toutes les gestions d'erreurs, les validations, les cas limites, les messages d'erreur explicites, la persistance de l'état quand ça plante — ça, c'est 2 semaines. »

« Donc quand une spec ne couvre que le cas nominal, elle ne couvre que 20% du comportement réel du système. »

Silence dans l'équipe.

Rafael : « Et pourtant on passe souvent en sprint planning sur des specs qui ne couvrent que ça. »

Jonas : « Exactement. Et on se demande pourquoi les estimations sont toujours dépassées. »

La règle des 80/20 des cas d'exception a une implication directe sur les estimations : une story dont la spec ne couvre que le cas nominal est estimée sur 20% de la complexité réelle. Les 80% restants seront "inventés" en cours de développement — parfois bien, parfois mal — et représenteront systématiquement un dépassement d'estimation.

Spécifier les cas d'exception n'est pas du sur-spec. C'est la condition d'une estimation juste.

3.4 Jonas et la « définition de terminé » pour chaque story

Jonas a développé une pratique personnelle qui complète le format IF…THEN… : avant de commencer le développement d'une story, il écrit lui-même les critères de test qu'il va utiliser pour la déclarer "terminée". Cette pratique — qu'il appelle "la

définition de terminé story-par-story" — lui permet de détecter les ambiguïtés résiduelles après la transformation IF…THEN…

Jonas — Début de sprint — Lundi matin

Jonas prend la story "Génération de rapport mensuel" et ouvre un nouveau document. En haut : "Définition de terminé — Story S23 — Génération de rapport mensuel."

Il commence à écrire les tests :

"Test 1 : En tant qu'utilisateur Standard, je génère un rapport mensuel sur janvier. Résultat attendu : rapport affiché en moins de 3 secondes avec les données de janvier."

"Test 2 : En tant qu'utilisateur Standard, je génère un rapport couvrant 12 mois. Résultat attendu : rapport affiché en moins de 3 secondes avec les données des 12 mois."

Il s'arrête. La spec dit "moins de 3 secondes pour des données allant jusqu'à 100 000 enregistrements". Il ne sait pas combien d'enregistrements couvrent "12 mois" dans les données de test. Il ne sait pas non plus si "les données de test" correspondent aux données réelles de production en termes de volume.

Il note une question : "Volume de données par mois dans les données de production — nécessaire pour valider que les 3 secondes sont réalistes."

Il continue. Arrivé au test 7 — le cas d'exception pour le service email indisponible — il réalise qu'il ne sait pas comment il va simuler l'indisponibilité du service email en test. La spec dit ce qui doit se passer, mais pas comment l'équipe QA va vérifier que ça se passe bien.

Il a 3 questions à poser avant de commencer. La story lui avait paru claire. Écrire les tests a révélé 3 zones grises.

La pratique de Jonas illustre le principe de testabilité comme révélateur d'ambiguïtés. Écrire les tests AVANT de coder force une réflexion sur le comportement exact attendu — une réflexion que le développement lui-même ne forcerait pas de façon aussi explicite.

Ce principe est lié au Test-Driven Development (TDD) — mais Jonas ne l'applique pas dans une logique de TDD strict. Il l'applique comme outil de détection d'ambiguïtés. Si les tests peuvent être écrits sans hésitation, la story est claire. Si les tests révèlent des zones grises, la story revient à Émilie pour clarification avant que le développement commence.

L'impact mesuré de la pratique

Jonas tient un suivi informel de l'impact de cette pratique sur ses 6 derniers mois dans l'équipe :

Indicateur	Avant la pratique	Après 6 mois de pratique
Questions de clarification posées en cours de développement	~4 par sprint	~0,5 par sprint
Stories retournées après démo pour non-conformité	~2 par sprint	~0,2 par sprint
Dépassements d'estimation significatifs (>50%)	~30% des stories	~8% des stories
Stories bloquées en QA pour comportement inattendu	~3 par sprint	~0,5 par sprint

Ces chiffres ne sont pas des données scientifiques — ils sont les observations personnelles de Jonas. Mais ils illustrent une tendance cohérente avec la logique sous-jacente : écrire les tests avant de coder réduit radicalement les coûts de correction tardive.

3.5 Le workshop IF…THEN… avec le client

La transformation des specs floues en IF…THEN… est un exercice que l'équipe peut faire en interne — et qui révèle les ambiguïtés internes. Mais certaines ambiguïtés ne peuvent pas être résolues sans le client. Elles nécessitent une décision métier que l'équipe n'a pas le mandat de prendre.

Pour ces ambiguïtés, Émilie a développé le workshop IF…THEN… avec le client : une session de travail en collaboration avec Anne, où les specs en format IF…THEN… sont lues et validées ensemble, clause par clause.

Salle de conférence client — Semaine 6 du projet Helios — Workshop de 3 heures

Émilie a imprimé les specs en format IF…THEN… pour les 5 fonctionnalités prioritaires du prochain sprint. Elle a surligné en jaune les clauses qui nécessitent une validation ou une décision de la part d'Anne.

Elle ouvre avec le protocole du workshop : « Aujourd'hui, je vais vous lire chaque spec en format IF…THEN…. Pour chaque clause, vous avez deux options : dire "c'est correct" et passer à la suite, ou dire "attendez" et on s'arrête pour discuter. L'objectif est de s'assurer que ce que j'ai écrit correspond exactement à ce que vous voulez que le système fasse. »

Anne : « D'accord. »

Émilie commence : « Fonctionnalité 1 : Alerte de conformité. Cas nominal — IF un indicateur de conformité dépasse le seuil configuré pour sa catégorie THEN le système... »

Anne : « Attendez. Le seuil configuré pour sa catégorie. Chaque indicateur a un seuil différent ? »

Émilie : « C'est ma compréhension, oui. Est-ce correct ? »

Anne : « Oui, mais... en réalité, certains indicateurs ont un seuil absolu et d'autres ont un seuil relatif — en pourcentage d'écart par rapport à la période

précédente. J'aurais dû préciser ça. »

Émilie note. Rafael, qui écoute en fond, se redresse. L'architecture qu'il avait prévue supposait des seuils absolus uniquement. Un seuil relatif implique de conserver les données de la période précédente — une contrainte de stockage différente.

Et ce n'est que la première clause de la première fonctionnalité.

Le workshop IF…THEN… avec Anne révèle 6 malentendus majeurs en 3 heures — des malentendus qui auraient normalement été découverts lors des démos de sprint, après plusieurs semaines de développement.

La valeur de ces 3 heures peut être calculée :

- Malentendu 1 (seuils relatifs vs absolus) : révélé en semaine 6 = 2 heures de discussion et d'ajustement de l'architecture. Révélé en semaine 18 = 3 semaines de refonte de l'architecture.
- Malentendu 2 (qui reçoit les alertes de conformité) : révélé en semaine 6 = 30 minutes de décision. Révélé en semaine 14 = 1 semaine de reconfiguration du système de notification.
- Malentendu 3 à 6 : économies similaires, proportionnelles à leur découverte en semaine 6 plutôt qu'en semaines 14-24.

Anne, en sortant du workshop : "C'est la réunion la plus utile qu'on ait eue depuis le début du projet. Je n'aurais jamais su expliquer ce que je voulais aussi précisément sans votre format de questions. Et honnêtement — j'ai découvert des choses que je n'avais moi-même pas clarifiées dans ma tête."

Protocole du workshop IF…THEN… client

Outil terrain — Protocole du workshop IF…THEN… client

Préparation (par l'analyste) :

1. Sélectionner les fonctionnalités prioritaires du prochain sprint (maximum 6-8 fonctionnalités par session).
2. Transformer les specs en format IF…THEN… complet (cas nominal + cas d'exception identifiés).
3. Surligner en jaune les clauses qui nécessitent une décision ou validation client.
4. Surligner en rouge les clauses qui contiennent des hypothèses de l'équipe.
5. Préparer une liste des questions ouvertes à poser si la lecture ne les révèle pas naturellement.

Animation (pendant la session) :

1. Expliquer le format IF…THEN… au client en 5 minutes (si première session).
2. Lire chaque spec lentement, clause par clause, en faisant une pause après chaque clause pour observation du client.
3. Pour chaque "attendez" du client : noter la question, discuter jusqu'à résolution, mettre à jour la spec sur le moment (ne pas "noter pour plus tard" — résoudre maintenant).
4. Pour chaque clause surlignée en rouge : exposer explicitement l'hypothèse de l'équipe et demander validation.
5. Résumer les décisions prises à la fin de la session et les faire valider à voix haute par le client.

Après la session :

1. Mettre à jour les specs avec toutes les décisions prises dans les 24 heures.
2. Envoyer un résumé des décisions au client pour validation écrite.

3. Mettre à jour le glossaire si de nouveaux termes ont été définis.

4. Partager les specs mises à jour avec toute l'équipe avant le sprint planning.

3.6 Anecdote — Nadia et le projet sauvé

Nadia n'était pas naturellement partisane du workshop IF…THEN… Au départ, elle le considérait comme "trop procédurier" et chronophage. C'est l'anecdote du projet Atlas — un projet complexe d'intégration ERP pour un client grand compte dans le secteur manufacturier — qui l'a convaincue de son caractère indispensable.

Projet Atlas — Semaine 2 — Après le premier workshop IF…THEN…

Nadia a accepté le workshop IF…THEN… à contrecœur — Émilie avait insisté, et le souvenir du projet PRISM était encore frais. Elle avait bloqué 3 heures dans l'agenda du client pour "validation des spécifications".

La session avait à peine commencé quand la première révélation était tombée.

Émilie lisait : « IF le système reçoit une mise à jour d'inventaire du système ERP THEN il synchronise les données en temps réel avec la base de données du portail client... »

Le responsable IT côté client, Jean-Paul, avait levé la main : « Qu'entendez-vous par "temps réel" ? »

Rafael avait répondu : « Synchronisation immédiate dès réception du signal — latence de moins de 500ms. »

Jean-Paul : « Mais l'ERP fait ses synchronisations en batch, toutes les 4 heures. Il n'y a pas de signal en temps réel. »

Silence.

Rafael : « Votre ERP ne supporte pas les webhooks ? »

Jean-Paul : « Il est en version 2019. Pas de webhooks. Synchronisation batch uniquement. »

Nadia calcule mentalement. Rafael avait conçu toute l'architecture autour d'une synchronisation event-driven en temps réel. Si l'ERP ne supporte que le batch, l'architecture doit être repensée de fond en comble — les connecteurs, la logique de réconciliation, la gestion de la cohérence des données pendant les fenêtres entre les batches...

En semaine 2, cette découverte nécessite 2 semaines de redéfinition de l'architecture. En semaine 18, elle aurait nécessité une refonte complète de 3 mois — avec l'infrastructure partiellement construite.

Nadia se tourne vers Émilie : « Bien. On annule ma réunion de cet après-midi. On continue le workshop. »

La découverte du batch ERP en semaine 2 était la plus significative — mais pas la seule. Le workshop Atlas a révélé en total 9 hypothèses techniques qui étaient fausses ou incomplètes. Toutes auraient été découvertes en cours de développement. Toutes auraient généré des coûts de correction exponentiellement plus élevés que les 3 heures du workshop.

Nadia tire la conclusion avec une clarté qui surprend son équipe lors de la rétrospective :

Nadia — Rétrospective — Fin de semaine 2

« J'ai passé ma carrière à croire que la vitesse de démarrage était le gage de succès d'un projet. Commencer vite, livrer vite, ajuster en cours de route. »

« Le projet PRISM m'a appris que livrer vite dans la mauvaise direction, ça ne

sert à rien. Le workshop IF…THEN… m'a appris que 3 heures de précision en semaine 2 valent plus que 3 mois de refonte en semaine 18. »

« Dorénavant, chaque projet démarre avec un workshop IF…THEN…. Non-négociable. »

Cette décision de Nadia — institutionnaliser le workshop IF…THEN… comme standard de démarrage — est la décision managériale qui transforme une pratique individuelle en pratique d'équipe. Sans ce mandat explicite, le workshop resterait dépendant de la volonté d'Émilie de l'animer et de la disponibilité du client. Avec ce mandat, il devient un livrable du projet au même titre que le plan de projet ou la charte de gouvernance.

📌 À retenir — Chapitre 3

- Le format IF…THEN… force l'explicitation du contexte (IF), du comportement attendu (THEN), des acteurs, des données et des conditions. Il rend les ambiguïtés **structurellement impossibles à ignorer**.
- Chaque spec floue transformée révèle en moyenne **3 à 5 décisions implicites** qui auraient été prises unilatéralement par le développeur. Ces décisions sont maintenant prises explicitement, avec le client.
- Les **cas d'exception** représentent 80% du comportement réel d'un système. Une spec qui ne couvre que le cas nominal est une spec à 20% de complétude — et une estimation à 20% de précision.
- Écrire les tests **avant** de coder (pratique de Jonas) révèle les zones grises résiduelles après la transformation IF…THEN…. C'est le test ultime de la testabilité d'une spec.
- Le **workshop IF…THEN… client** est l'investissement qui a le retour sur

investissement le plus élevé dans l'ensemble du Guide Jaune : 3 heures en semaine 2 évitent régulièrement plusieurs semaines de refonte en fin de projet.

- Le workshop IF…THEN… doit être **institutionnalisé** par la direction de projet — pas laissé à l'initiative individuelle de l'analyste.

🔍 Symptômes à observer dans votre équipe

- Les développeurs posent des questions de clarification en cours de sprint ("est-ce que le système doit aussi gérer le cas où… ?") — signal que les cas d'exception n'ont pas été spécifiés.
- Les estimations sont systématiquement dépassées — signal que la spec ne couvre que le cas nominal et que les cas d'exception sont "inventés" en cours de développement.
- Les démos révèlent des "mais qu'est-ce qui se passe si… ?" de la part du client que personne n'avait pensé à couvrir.
- Les tests QA bloquent des stories pour des comportements non définis dans la spec ("ce n'est pas documenté — qu'est-ce qu'on est censé faire ?").
- Les specs sont toutes rédigées au présent narratif ("le système envoie une confirmation") sans format IF…THEN….
- Il n'y a pas de workshop de validation client des specs avant le démarrage du développement.

💬 Questions de réflexion — Chapitre 3

1. Prenez une spec récente de votre équipe et essayez de la transformer en

format IF…THEN… complet (cas nominal + 3 cas d'exception). Quelles zones grises la transformation révèle-t-elle ?

2. Pour votre prochain sprint : demandez à chaque développeur d'écrire les tests de sa story AVANT de commencer le développement. Quelles questions remontent ?

3. Avez-vous déjà organisé un workshop de validation des specs avec votre client en format lecture commentée ? Si oui, qu'est-ce que ça a révélé ? Si non, quelle serait la résistance à surmonter pour l'organiser ?

4. Dans votre projet actuel, quelle fonctionnalité aurait le plus à gagner d'une transformation IF…THEN… complète avec tous les cas d'exception ? Pourquoi ?

5. Si vous étiez Nadia, à quel moment de votre projet aurait-il été le plus bénéfique de tenir un workshop IF…THEN… avec le client ?

⚠️ Signal de vigilance — Chapitre 3

L'hypothèse technique non documentée : chaque fois qu'un développeur ou un architecte fait une hypothèse sur le comportement d'un système tiers, d'une API, d'un protocole, ou d'une contrainte d'infrastructure — et que cette hypothèse n'est pas validée avec le client ou l'équipe technique cliente — c'est une bombe à retardement. Les hypothèses techniques non documentées sont particulièrement dangereuses parce qu'elles sont invisibles dans la spec (qui parle du comportement fonctionnel) et ne remontent que quand le système est en développement avancé. Le workshop IF…THEN… avec la participation de l'équipe technique cliente (comme Jean-Paul dans le projet Atlas) est le seul outil qui révèle ces hypothèses à temps.

CHAPITRE 4

La culture de la question : créer un environnement où les ambiguïtés remontent

> *« Une question posée à temps est un acte de courage professionnel. Une ambiguïté tu sous silence est un acte de lâcheté organisationnelle. »*

Les outils des chapitres précédents sont puissants. L'audit de spécification, le glossaire de projet, le format IF…THEN…, le workshop client — ce sont des outils concrets, testés sur le terrain, dont l'efficacité est documentée. Mais ces outils ne fonctionnent que dans un contexte culturel qui les supporte.

Ce contexte culturel — c'est une équipe où les ambiguïtés remontent naturellement, où poser une question de clarification est perçu comme un acte de professionnalisme et non comme un aveu d'ignorance, où la précision est valorisée autant que la vélocité. C'est une équipe où Émilie peut dire "j'ai 7 ambiguïtés dans ces stories" sans craindre la réponse "on verra en développement".

Ce contexte culturel ne s'installe pas par décret. Il se construit délibérément, avec des pratiques spécifiques, des rituels formels, et un changement de posture managériale. C'est ce que ce chapitre adresse.

4.1 Pourquoi les équipes ne posent pas les questions

Le chapitre 1 a identifié les raisons individuelles pour lesquelles Émilie et Jonas ne posaient pas leurs questions. Ce chapitre approfondit l'analyse au niveau organisationnel — parce que les raisons individuelles sont souvent des symptômes de dynamiques organisationnelles plus profondes.

La dynamique de statut dans les réunions client

Dans une réunion avec un client grand compte, la dynamique de statut est souvent asymétrique. Le client est la personne qui paie. L'équipe est la prestataire. Cette relation crée implicitement une pression pour "paraître compétent" devant le client — c'est-à-dire, paradoxalement, pour ne pas poser de questions.

Dans cette dynamique, poser une question qui révèle une lacune de compréhension peut être perçu comme une menace pour la crédibilité de l'équipe. "Si je leur demande ce que signifie 'réconciliation des données', vont-ils se demander si j'ai l'expérience suffisante pour ce projet ?" Cette peur — non fondée mais très réelle — pousse les membres de l'équipe à faire des hypothèses plutôt qu'à poser des questions.

La réalité est exactement inverse : un prestataire qui pose des questions précises de clarification démontre une compréhension sophistiquée du domaine, une rigueur professionnelle, et un engagement envers la qualité du résultat. Les clients les plus expérimentés préfèrent un prestataire qui pose beaucoup de questions en début de projet à un prestataire qui livre en fin de projet quelque chose qui ne correspond pas à leurs attentes.

Mais cette réalité doit être explicitée — par Nadia, en présence d'Anne. Ce qui n'est pas dit n'existe pas dans la culture de l'équipe.

La pression implicite de la vélocité

Dans un contexte agile, la vélocité est une métrique visible et valorisée. Le nombre de story points livrés par sprint est mesuré, discuté, comparé. Ce que cette métrique ne capture pas : le temps passé en amont à clarifier les specs — temps

qui réduit le nombre de points livrés dans le sprint de clarification, mais augmente la qualité et la précision de tout ce qui suit.

Quand la vélocité est la seule métrique visible, clarifier les specs devient une activité perçue comme "non productive" — un temps qui ne génère pas de points livrés. Cette perception crée une pression implicite pour "avancer" plutôt que pour "clarifier". Et cette pression est particulièrement forte sur les analystes (dont le travail de clarification n'est pas capturé dans les story points) et sur les juniors (qui craignent d'être perçus comme des freins à l'avancement).

La solution : rendre la clarification visible dans les métriques. Les ambiguïtés détectées et résolues avant le sprint sont une métrique aussi précieuse que les story points livrés. Les workshops IF…THEN… ont une durée et un résultat mesurable. Le temps investi en élicitation structurée peut être mis en regard du temps économisé en corrections.

L'absence de canal formel

Il n'y a pas d'ambiguïté dans la structure organisationnelle d'une équipe sur qui est responsable de livrer du code (le développeur), qui est responsable de tester (le QA), qui est responsable de gérer le projet (le chef de projet). Mais qui est responsable de remonter les ambiguïtés ? Qui est responsable de s'assurer que chaque terme clé est défini avant le démarrage du développement ?

Si cette responsabilité n'est pas explicitement assignée, elle appartient à tout le monde — ce qui signifie qu'elle appartient à personne. Chacun suppose que quelqu'un d'autre s'en occupe. Et les ambiguïtés restent dans les carnets personnels jusqu'à la démo.

La clarté de rôles — aussi précise que dans une spec IF…THEN… — est nécessaire pour que les ambiguïtés remontent de façon systématique.

La peur de la réaction managériale

La dernière raison — et peut-être la plus déterminante — est la peur de la réaction du manager. Si Émilie a remonté des ambiguïtés dans le passé et a reçu comme réponse "on n'a pas le temps pour ça, on verra en développement", elle a appris

que remonter des ambiguïtés ne sert à rien et génère de la friction. Elle a intégré ce signal et a arrêté de remonter ses ambiguïtés.

La transformation de Nadia — de "on avance" à "les ambiguïtés sont prioritaires" — est donc une condition sine qua non pour que les pratiques des chapitres 2 et 3 fonctionnent. Les outils sans le changement de culture managériale sont des outils sans environnement porteur. Ils seront utilisés une ou deux fois, puis abandonnés à la prochaine pression de délai.

4.2 Le protocole d'ambiguïté — outil terrain

Pour qu'une ambiguïté remonte de façon systématique, elle a besoin d'un canal formel. Ce canal, c'est le **protocole d'ambiguïté** — un ensemble de règles simples et de structures visibles qui donnent aux ambiguïtés un statut aussi réel et aussi urgent que les bugs ou les user stories.

Composante 1 — Le registre d'ambiguïtés

Le registre d'ambiguïtés est un artefact vivant, visible de toute l'équipe, qui log chaque ambiguïté détectée. Il peut prendre la forme d'une colonne dédiée dans le backlog (dans Jira, Azure DevOps, ou équivalent) ou d'un document partagé avec une structure simple.

Pour chaque ambiguïté dans le registre :

Champ	Description	Exemple
ID	Identifiant unique de l'ambiguïté	AMB-047
Story liée	La story ou l'exigence concernée	S23 — Génération de rapport mensuel
Description	Description précise de l'ambiguïté en une phrase	"Le volume de données par mois en production n'est pas connu — impossible de valider le critère de 3 secondes."

Champ	Description	Exemple
Type	Type d'ambiguïté selon la taxonomie du chapitre 1	Temporelle / De portée
Détecteur	Qui a détecté l'ambiguïté	Jonas
Owner	Qui est responsable de la résoudre	Émilie (à clarifier avec Anne)
Deadline	Avant quelle cérémonie doit-elle être résolue	Avant le sprint planning du [date]
Statut	Ouverte / En cours / Résolue / Escaladée	En cours
Résolution	La décision prise pour résoudre l'ambiguïté	"Volume moyen par mois : 12 000 lignes selon Anne. Critère de 3 secondes maintenu."

Composante 2 — La règle de blocage

Une ambiguïté non résolue dans le registre bloque la story concernée lors du sprint planning. Il n'y a pas de dérogation possible sans validation explicite de Nadia ET d'Émilie. Cette règle de blocage est la plus importante du protocole — c'est elle qui donne aux ambiguïtés leur urgence réelle.

Sans règle de blocage, le registre d'ambiguïtés est une liste de souhaits. Avec la règle de blocage, c'est un système de contraintes qui force la résolution avant que le développement commence.

Composante 3 — La revue d'ambiguïtés hebdomadaire

Chaque semaine, l'équipe tient une "revue d'ambiguïtés" de 15 à 30 minutes (selon le volume d'ambiguïtés en cours). Cette revue :

- Passe en revue toutes les ambiguïtés ouvertes du registre.
- Valide les résolutions proposées.
- Escalade les ambiguïtés dont la deadline approche sans résolution.
- Identifie les nouvelles ambiguïtés détectées depuis la dernière revue.

La revue d'ambiguïtés n'est pas une réunion supplémentaire dans un agenda déjà chargé — elle peut être intégrée en début de backlog grooming ou de sprint planning. Sa durée est inversement proportionnelle à la maturité du processus de spécification : plus les specs sont produites avec qualité en amont, moins il y a d'ambiguïtés ouvertes à gérer.

Composante 4 — La reconnaissance du détecteur

Une pratique simple mais efficace : reconnaître publiquement les membres de l'équipe qui détectent des ambiguïtés significatives avant qu'elles ne coûtent quelque chose. "Merci, Jonas, l'ambiguïté AMB-047 que tu as remontée nous a évité une semaine de refonte" — cette phrase, dite par Nadia lors de la rétrospective, change la culture.

Elle signale que détecter et remonter une ambiguïté est un acte valorisé — pas un aveu d'ignorance, pas un frein à l'avancement. C'est une contribution mesurable à la qualité du projet.

4.3 Émilie reprend sa voix

Salle de réunion — Revue d'ambiguïtés — Semaine 8 du projet Helios

Émilie ouvre la réunion. Sur l'écran : le registre d'ambiguïtés du projet Helios. 7 lignes. 7 ambiguïtés ouvertes dans les stories candidates pour le prochain sprint.

Nadia, en entrant dans la salle : « 7 ? On a autant d'ambiguïtés non résolues ? »

Émilie : « J'en ai détecté 7 en lisant les stories. Trois dans la fonctionnalité de workflow d'approbation, deux dans le module de reporting, deux dans la gestion des utilisateurs. »

Nadia s'assoit. Elle regarde le registre. Elle regarde Émilie. « Tu les voyais

depuis le début ? »

Émilie : « Oui. »

Nadia, après un moment : « Et avant le projet PRISM — tu les voyais aussi ? »

Émilie : « Oui. Mais je ne savais pas que c'était mon rôle de les remonter formellement. Et je craignais de ralentir le projet. »

Silence dans la salle. Rafael, qui était entré en retard, s'est arrêté dans l'embrasure de la porte.

Nadia : « Dorénavant, c'est le rôle de tout le monde. De toi, de Jonas, de Rafael. Si quelqu'un voit une ambiguïté et ne la remonte pas, ce n'est pas de la discrétion — c'est du sabotage involontaire. »

Elle prend le marqueur et écrit sur le tableau blanc : "Une ambiguïté détectée et remontée = une démo réussie de plus."

Émilie, doucement : « Je préfère cette règle à l'ancienne. »

Nadia : « Moi aussi. Passons en revue les 7. »

La scène d'Émilie qui reprend sa voix est un moment charnière dans la transformation de l'équipe. Ce n'est pas seulement Émilie qui change — c'est Nadia qui reconnaît sa propre responsabilité dans le silence d'Émilie. Et cette reconnaissance est nécessaire. Le changement de culture ne peut pas venir uniquement de la base — il doit venir aussi du sommet.

Nadia fait trois choses dans cette scène qui changent la culture :

Elle normalise la détection d'ambiguïtés. "C'est le rôle de tout le monde" — pas seulement de l'analyste, pas seulement de la personne désignée. Tout le monde a le droit et le devoir de remonter ce qu'il voit.

Elle lie l'action à la conséquence positive. "Une ambiguïté détectée et remontée = une démo réussie de plus" — en reliant l'action au résultat positif

qu'elle prévient, Nadia change l'interprétation de l'ambiguïté remontée. Ce n'est plus un signal de problème. C'est un signal de professionnalisme.

Elle reconnaît sa propre responsabilité. Implicitement, dans sa question "Et avant le projet PRISM, tu les voyais aussi ?", Nadia reconnaît qu'elle a contribué au silence d'Émilie. Cette reconnaissance est rare et précieuse — elle crée un espace de confiance dans lequel Émilie peut répondre honnêtement.

Le suivi à 3 mois

Trois mois après l'introduction du registre d'ambiguïtés, Émilie fait un bilan :

Indicateur	Avant le registre	3 mois après
Ambiguïtés remontées par sprint	0 (officiellement)	6-9 par sprint
Ambiguïtés détectées en démo	3-5 par démo	0-1 par démo
Stories retournées après démo	2-3 par démo	0-0.5 par démo
Membres de l'équipe ayant soumis une ambiguïté	Émilie uniquement	Toute l'équipe (Émilie, Jonas, Rafael, 2 devs juniors)

Ces chiffres illustrent une transformation réelle : les ambiguïtés n'ont pas disparu — elles remontent maintenant au bon moment, par les bonnes personnes, avec un processus de résolution structuré. Ce n'est plus une loterie. C'est un système.

4.4 Anne comme partenaire de clarté

Jusqu'à présent, Anne a été présentée principalement dans son rôle de "commanditaire" — celle qui exprime les besoins et valide les livrables. Le Guide Jaune propose une transformation de ce rôle : Anne comme "**partenaire de définition**". Ce n'est pas un simple changement de vocabulaire. C'est un changement de posture qui a des implications concrètes sur comment le projet fonctionne.

La différence entre commanditaire et partenaire de définition

Dimension	Rôle de commanditaire	Rôle de partenaire de définition
Moment d'engagement	En début de projet (expression des besoins) et en fin (validation des livrables)	Tout au long du projet, à chaque point de clarification nécessaire
Nature de la contribution	Fournit les spécifications (ou les valide)	Co-construit les spécifications avec l'équipe
Rapport aux ambiguïtés	N'en est pas informé jusqu'à la démo	Est alerté dès la détection et participe à la résolution
Rapport aux cas d'exception	Les découvre lors des démos ou en production	Les définit en amont lors des sessions d'élicitation et des workshops
Rôle dans le glossaire	Reçoit le glossaire pour information	Participe à la définition et valide chaque entrée du glossaire

Éduquer Anne sur la valeur de l'investissement

Le changement de rôle d'Anne nécessite son adhésion — et son adhésion nécessite qu'elle comprenne pourquoi cet investissement de temps de sa part génère de la valeur pour elle.

Nadia organise une réunion de cadrage avec Anne en début de projet Helios. L'agenda : présenter le coût des malentendus non adressés (projet PRISM, anonymisé) et présenter le nouveau mode de travail avec l'équipe. Elle utilise un argument simple :

Nadia — Réunion de cadrage avec Anne — Semaine 1 du projet Helios

« Anne, j'ai besoin de vous expliquer quelque chose que nous avons appris à nos dépens. Dans un projet précédent similaire au vôtre, nous avions livré un système techniquement excellent, dans les délais et le budget — et qui ne

correspondait pas à ce que le client attendait vraiment. »

« La raison : des mots ambigus dans la spec initiale que personne n'avait clarifiés. Des réunions de validation qui se passaient avec un intermédiaire, pas avec le décideur. Des workshops qui auraient dû avoir lieu en semaine 2 et qui n'ont eu lieu qu'en semaine 40. »

« Ce projet m'a coûté une relation client et 6 mois de travail supplémentaire. »

« Dans votre projet, je vous demande quelque chose de différent. Je vous demande 4 à 6 heures de votre temps par mois — en sessions structurées, avec un agenda précis — pour co-définir avec nous exactement ce que le système doit faire. Pas juste valider ce que nous avons compris, mais construire avec nous la compréhension commune. »

Anne : « Et ça change quoi ? »

Nadia : « Ça change tout. Les questions que nous vous poserons en mois 2 sont des questions qui auraient autrement été répondues par nos développeurs — seuls, sans vous — en mois 6. Et dans mois 6, elles vous auraient coûté des semaines de refonte. »

Anne réfléchit. « 4 à 6 heures par mois. »

Nadia : « En échange de livraisons qui correspondent à ce que vous voulez vraiment. »

Anne : « D'accord. »

Cette conversation — explicite, directe, avec un exemple concret et une proposition claire — est la fondation de la relation de partenariat avec Anne. Elle pose les attentes dès le départ. Elle explique pourquoi l'investissement est demandé. Et elle donne à Anne le cadre pour comprendre que ses 4 à 6 heures mensuelles ne sont pas une charge — elles sont la meilleure assurance contre un résultat décevant.

Ce que la relation de partenariat change en pratique

Avec Anne comme partenaire de définition, plusieurs choses changent concrètement :

Les décisions métier sont prises par Anne, pas interprétées par l'équipe. Quand une ambiguïté nécessite une décision métier (quel est le seuil d'approbation automatique ? Quels utilisateurs ont accès à quel module ?), Émilie contacte Anne directement et obtient une réponse dans les 48 heures — parce que le processus a été établi dès le début.

Anne est présente aux démos — pas un intermédiaire. Les démos de sprint sont les moments où la compréhension de l'équipe et les attentes du client sont confrontées. Si un intermédiaire représente Anne, les feedbacks sont filtrés et retardés. La présence directe d'Anne est une règle non négociable du projet Helios.

Le glossaire est signé par Anne. Chaque entrée du glossaire porte la validation d'Anne. Si lors d'une démo elle dit "ce n'est pas ce que je voulais dire", la question est posée : "La définition dans le glossaire est-elle différente de votre attente ?" Si oui, le glossaire est mis à jour. Si non, la spec a été mal implémentée — et ce n'est pas un malentendu de définition.

4.5 « Ce qui n'a pas fonctionné » — La première tentative

Le parcours de transformation de l'équipe n'a pas été linéaire. Il y a eu des tentatives infructueuses, des résistances, des moments où les nouvelles pratiques ont failli être abandonnées. L'honnêteté du Guide Jaune exige de documenter ces moments — parce que les organisations qui réussissent leur transformation ne sont pas celles qui évitent les obstacles, mais celles qui apprennent à les traverser.

Six mois avant le projet Helios — Émilie, première tentative

Émilie avait découvert le format IF…THEN… dans un article de pratiques agiles. Elle avait passé un weekend à restructurer les specs du projet en cours. Le lundi matin, elle avait présenté son travail à Nadia.

Nadia avait parcouru les deux premières pages du document restructuré. Elle avait levé les yeux.

Nadia : « C'est très bien, Émilie. Mais on n'a pas le temps pour ça. On est en semaine 8. Le sprint 4 commence demain. »

Émilie : « Je sais. Mais si on ne clarifie pas ces ambiguïtés maintenant... »

Nadia : « On verra en développement. Si Jonas a des questions, il te contacte. »

Émilie avait rangé son document. Elle ne l'avait plus ressorti. Jusqu'au projet PRISM.

La première tentative d'Émilie avait échoué pour trois raisons :

Raison 1 — Le mauvais moment. Introduire une nouvelle pratique en semaine 8 d'un projet existant, quand l'élan et les habitudes sont établis, est beaucoup plus difficile qu'en semaine 1. La résistance au changement est proportionnelle au momentum existant. La pratique IF…THEN… devait être introduite dès le départ — pas en cours de route.

Raison 2 — L'absence de preuve de valeur. Nadia n'avait pas encore vécu le projet PRISM. Elle n'avait pas la douleur suffisante pour justifier l'investissement que la nouvelle pratique demandait. Sans preuve de valeur, toute nouvelle pratique est vécue comme un coût sans bénéfice visible.

Raison 3 — L'initiative solitaire sans soutien managérial. Émilie portait seule le changement. Sans le soutien de Nadia — sans le mandat explicite de "cette pratique est maintenant standard dans notre équipe" — la nouvelle pratique était une proposition individuelle fragile, facilement écartée par la pression de l'urgence.

Ce que la deuxième tentative a fait différemment

La deuxième tentative — celle qui a fonctionné, dans le contexte du projet Helios — a bénéficié de trois conditions nouvelles :

Condition 1 — La preuve par la douleur. Le projet PRISM avait fourni à Nadia l'expérience directe du coût des malentendus. Elle n'avait plus besoin d'être convaincue intellectuellement — elle l'avait vécu. Cette expérience avait transformé sa posture de "on verra en développement" à "on clarifie avant".

Condition 2 — L'introduction au bon moment. Helios avait commencé avec les nouvelles pratiques dès la semaine 1. L'audit de spécification, le glossaire, les sessions d'élicitation, le workshop IF…THEN… — tout était en place avant le premier sprint de développement. Il n'y avait pas de momentum d'ancienne habitude à surmonter.

Condition 3 — Le mandat managérial explicite. Nadia avait expliqué à toute l'équipe, lors du kick-off du projet Helios, que ces nouvelles pratiques n'étaient pas optionnelles. Elle avait expliqué pourquoi, avec l'histoire du projet PRISM (anonymisée). Elle avait clairement signalé que remonter des ambiguïtés était désormais une attente — pas une option.

La leçon de ces deux tentatives est à la fois humbling et libératrice : parfois il faut la preuve par la douleur avant que la prévention soit acceptée. Cette réalité n'est pas une défaillance humaine — c'est la nature du changement organisationnel. Les organisations n'investissent pas dans la prévention de problèmes qu'elles n'ont pas encore vécus. Une fois qu'elles les ont vécus, elles comprennent l'investissement.

Ce qui est dans le contrôle de chacun dans une équipe, c'est d'être prêt — d'avoir les outils, les pratiques, et la conviction — pour que quand la preuve par la douleur arrive, la réponse soit immédiate et cohérente.

Émilie était prête. Et quand le projet PRISM a créé l'ouverture, elle a su quoi proposer.

Ce qui n'a pas fonctionné — Le glossaire imposé (suite de 2.3)

Dans le même projet où Émilie avait tenté d'introduire le format IF…THEN…, elle avait aussi produit un glossaire unilatéralement — sans session de négociation avec le client. Ce glossaire avait été distribué à l'équipe et au client comme "le vocabulaire officiel du projet".

Résultat : personne ne l'avait utilisé. L'équipe pensait que les définitions étaient les décisions d'Émilie, pas les leurs. Le client trouvait certaines définitions incorrectes par rapport à son vocabulaire métier — mais comme il n'avait pas été invité à les définir, il ne se sentait pas légitime pour les contester.

Deux mois plus tard, une réunion sur le terme "réconciliation" avait révélé que personne n'utilisait la définition du glossaire. Chacun avait sa propre définition. Le glossaire était un document mort.

Leçon confirmée : le processus de négociation EST le glossaire. Le document est sa trace. Sans le processus, le document ne vaut rien.

Écho DREAM-TEAM — Karim sur la culture de la question

Karim raconte : « Quand je suis arrivé chez ORIGINALIUM, j'avais peur de poser des questions devant le client. Je pensais que ça me ferait paraître moins compétent. »

« Théo m'a dit quelque chose qui a tout changé : "Karim, chez nous, la question est la compétence. Un QA qui ne pose pas de questions avant de tester n'est pas un QA — c'est un exécutant." »

« Maintenant, je pose toutes mes questions. Et j'observe que les clients les plus sophistiqués — ceux qui ont déjà eu des projets mal livrés — sont les plus reconnaissants quand je pose des questions précises. Ils ont appris à quoi coûte l'absence de questions. »

Rania ajoute : « La culture de la question ne se décrète pas. Elle se montre. Chaque fois que Théo pose une question "naïve" en réunion client — une question dont tout le monde connaît la réponse sauf peut-être le client — il envoie un signal à toute l'équipe : les questions sont les bienvenues ici. »

📌 À retenir — Chapitre 4

- Les raisons pour lesquelles les équipes ne posent pas leurs questions sont **systémiques** : dynamique de statut, pression de la vélocité, absence de canal formel, peur de la réaction managériale. Aucune de ces raisons ne disparaît par la seule volonté individuelle.
- Le **protocole d'ambiguïté** — registre visible, règle de blocage, revue hebdomadaire, reconnaissance du détecteur — transforme la gestion des ambiguïtés d'une initiative individuelle en pratique systémique.
- La transformation d'Émilie (de l'analyste silencieuse à l'analyste qui remonte tout) n'est possible que parce que **Nadia change aussi**. Le changement de culture nécessite un changement de posture managériale.
- Anne n'est pas une commanditaire à satisfaire — elle est une **partenaire de définition** à engager. Ce changement de posture nécessite une éducation explicite sur la valeur de son investissement de temps.
- La première tentative échoue souvent. Ce n'est pas une raison d'abandonner — c'est une information sur les conditions nécessaires au succès. **La preuve par la douleur** crée l'ouverture. Les outils préparés à l'avance remplissent l'ouverture.
- La culture de la question se **montre** avant de se décréter. Chaque fois qu'un senior pose une question "naïve" en réunion, il donne la permission aux

juniors de faire de même.

🔍 Symptômes à observer dans votre équipe

- Les membres de l'équipe attendent la fin des réunions pour poser leurs vraies questions — entre eux, jamais devant le client.
- Les ambiguïtés détectées restent dans les carnets personnels et les notes privées — elles n'ont aucun endroit visible où aller.
- Le chef de projet répond "on verra" ou "on avance" quand une ambiguïté est soulevée — signal fort d'une culture qui décourage la précaution.
- Les démos avec le client sont des moments de tension — chacun espère en secret que "ça passera" — plutôt que des moments de validation confiante.
- La relation avec le client est principalement gérée par une seule personne (le chef de projet) — les autres membres de l'équipe n'ont pas accès direct aux décideurs métier.
- Il n'existe pas de registre ou de liste visible des ambiguïtés en cours dans le projet.

💬 Questions de réflexion — Chapitre 4

1. Quel est le signal le plus fort que votre équipe envoie sur la question des questions ? Est-ce que poser une question de clarification est valorisé ou perçu comme un frein ?
2. Si vous êtes manager ou chef de projet : pensez à la dernière fois qu'un membre de l'équipe a soulevé une ambiguïté. Quelle a été votre réponse ? Qu'est-ce que cette réponse a signalé à toute l'équipe sur la culture des

questions ?

3. Si vous êtes analyste, développeur ou QA : y a-t-il des ambiguïtés actuellement dans votre carnet que vous n'avez pas remontées ? Pourquoi ?

4. Votre client actuel joue-t-il un rôle de commanditaire ou de partenaire de définition ? Qu'est-ce qui changerait dans la relation si vous proposiez le modèle de partenariat décrit dans ce chapitre ?

5. Avez-vous vécu votre propre "première tentative" d'une pratique de clarification qui a échoué ? Quelles étaient les conditions manquantes ? Sont-elles présentes aujourd'hui ?

⚠️ Signal de vigilance — Chapitre 4

Le test de la rétrospective : lors de votre prochaine rétrospective, posez cette question à l'équipe : "Y a-t-il des ambiguïtés que vous avez détectées pendant ce sprint et que vous n'avez pas remontées ?" Si la réponse est oui — et elle le sera dans la majorité des équipes qui posent cette question pour la première fois — vous avez mesuré l'étendue de votre problème culturel. La bonne nouvelle : des équipes qui peuvent répondre honnêtement à cette question sont des équipes prêtes à changer. La mauvaise nouvelle : des équipes où personne ne répond à cette question ont un problème de confiance psychologique qui précède le problème de spécification.

CONCLUSION DE LA PARTIE A

Le diagnostic est posé — La clarté commence maintenant

> *« Comprendre pourquoi quelque chose ne fonctionne pas est déjà la moitié de la solution. L'autre moitié, c'est d'agir différemment. »*

La Partie A du Guide Jaune a fait un travail de diagnostic. Un diagnostic complet, honnête, parfois inconfortable — parce qu'il ne pointe pas vers des boucs émissaires ou des situations exceptionnelles. Il pointe vers des mécanismes ordinaires, des comportements courants, des systèmes de communication inadaptés qui produisent des malentendus avec la même régularité qu'une machine produit ses pièces.

Voici ce que la Partie A a établi.

Ce que nous savons maintenant

Les malentendus sont structurels. Ils ne viennent pas de personnes négligentes ou de mauvaises intentions. Ils viennent de cadres de référence différents, de chaînes de traduction qui perdent de l'information à chaque maillon, et d'implicites culturels que personne n'a jamais explicités. Le projet PRISM n'est pas un accident. C'est le résultat prévisible d'un système de communication qui n'a pas été conçu pour aligner des mondes différents.

Les ambiguïtés ont une taxonomie. Elles ne sont pas toutes identiques, et elles ne se traitent pas toutes de la même façon. L'ambiguïté lexicale appelle le glossaire. L'ambiguïté structurelle appelle la transformation IF…THEN…. L'ambiguïté de portée appelle la session d'élicitation structurée. L'ambiguïté temporelle appelle

la mesure. L'ambiguïté conditionnelle appelle la matrice des cas d'exception. Avoir un nom pour chaque type d'ambiguïté, c'est avoir la moitié de la solution.

Les outils existent. L'audit de spécification en 4 dimensions révèle le niveau de maturité d'une spec avant que le développement commence. Les 7 mots dangereux permettent une détection rapide des zones à risque. Le glossaire de projet transforme un vocabulaire ambigu en accord négocié. La session d'élicitation structurée en 4 temps (contexte, scénario nominal, cas d'exception, définition du succès) révèle les exigences implicites que personne n'avait formalisées. Le format IF…THEN… force la précision là où la prose permettait l'ambiguïté. Le workshop client valide les specs avec la source d'autorité — avant que le développement commence.

La culture est le précondition. Tous ces outils fonctionnent dans un contexte culturel qui les supporte. Un contexte où poser des questions est valorisé, où remonter une ambiguïté est un acte de professionnalisme, où le client est un partenaire de définition et non un destinataire passif. Sans ce contexte, les outils sont des procédures vides que personne n'utilisera sous pression.

Le changement commence par la douleur — puis par la décision. Émilie avait les notes. Nadia avait l'intention. Rafael avait les frustrations. Jonas avait les instincts. Ce qui leur manquait : le moment de cristallisation où toutes ces pièces s'assemblent en un système cohérent. Le projet PRISM a été ce moment. Et leur réponse — construire délibérément un système de clarté à partir des leçons de cet échec — est le modèle de transformation que le Guide Jaune documente.

Ce que la Partie A ne fait pas encore

La Partie A pose le diagnostic. Elle introduit les premiers outils. Elle illustre comment ces outils ont commencé à transformer l'équipe du projet Helios. Mais elle ne couvre pas encore l'intégration complète de ces pratiques dans le cycle projet — du premier jour à la dernière livraison.

Comment intégrer l'élicitation structurée dans la phase ÉLICITER du SDLC réconcilié — pas comme une réunion isolée, mais comme une pratique continue qui évolue avec la compréhension du projet ? Comment maintenir la clarté quand le contexte change, quand les interlocuteurs tournent, quand les priorités évoluent en cours de projet ? Comment construire une "spécification vivante" — un artefact qui reflète en permanence la compréhension partagée la plus récente, et non la compréhension initiale figée dans un document de 47 pages ? Comment mesurer, à l'échelle d'un portefeuille de projets, l'impact des pratiques de clarification sur la qualité des livrables et la satisfaction client ?

Ce sont les questions de la Partie B.

Le pont vers la Partie B

La Partie B — *Spécifier pour construire juste : les pratiques du SDLC réconcilié* — prend le relais là où la Partie A s'arrête. Elle approfondit l'intégration des pratiques de clarification dans chacune des six phases du SDLC réconcilié :

- **ÉLICITER :** comment structurer la relation avec le client dès le premier contact pour que la clarté soit un engagement mutuel dès le départ.
- **DÉFINIR :** comment construire une spécification vivante — un glossaire évolutif, des specs IF…THEN… versionnées, un registre d'ambiguïtés intégré au backlog.
- **PLANIFIER :** comment intégrer les workshops de clarification dans le plan de projet — avec des livrables, des participants, et des critères de succès — sans que ce soit perçu comme "du temps perdu".
- **IMPLÉMENTER :** comment maintenir la clarté pendant le développement — gestion des ambiguïtés émergentes, pratique du "test avant de coder", communication proactive des décisions d'implémentation.

- **DIFFUSER :** comment préparer les démos de façon à ce qu'elles valident la compréhension partagée plutôt que de révéler les malentendus — et comment transformer les feedbacks de démo en specs précises pour la suite.
- **CONFIRMER :** comment la grande boucle CONFIRMER → ÉLICITER alimente en permanence la clarté — comment chaque fin de projet est une source d'apprentissage pour améliorer les pratiques de spécification des projets suivants.

La Partie B introduit également des outils complémentaires : la cartographie des parties prenantes pour identifier qui a quelle autorité de décision sur quelle ambiguïté, le modèle de "spécification par couches" pour gérer des projets où la complexité métier est trop grande pour être spécifiée intégralement en amont, et les indicateurs de maturité de la spécification à l'échelle d'un portefeuille de projets.

Un dernier mot — Sur la couleur jaune

Le Guide Jaune s'appelle ainsi parce que la clarté est une forme de lumière. Et la lumière ne se décrète pas — elle se produit.

Émilie a produit de la lumière quand elle a ouvert son carnet de notes ce vendredi soir et compté ses douze coches rouges. Nadia a produit de la lumière quand elle a accepté que sa culture du "on avance" contribuait aux démos catastrophiques. Jonas a produit de la lumière quand il a commencé à écrire ses tests avant son code. Rafael a produit de la lumière quand il a nommé la règle qu'il s'était donnée : "Un terme non défini dans le glossaire est une spécification qui sera implémentée différemment à chaque fois."

Et Anne a produit de la lumière quand elle a accepté d'être partenaire de définition — quand elle a compris que ses 4 à 6 heures mensuelles n'étaient pas un coût, mais l'investissement qui garantissait que le système livré serait le système qu'elle voulait vraiment.

La clarté n'est pas un état final. C'est une pratique quotidienne. Elle se maintient par des questions posées à temps, par des termes définis ensemble, par des specs qui forcent la précision, par une culture qui valorise la rigueur autant que la vitesse.

La chaîne est immuable : **Clarté → qualité → confiance → excellence durable.**

Elle commence ici. Elle commence maintenant. Elle commence avec la prochaine spec que vous allez lire — et la première question que vous allez oser poser.

📌 Les 5 engagements de la Partie A

Avant de passer à la Partie B, voici les cinq engagements concrets que la Partie A vous invite à prendre pour votre prochain projet :

1. **Auditer.** Appliquer l'audit de spécification en 4 dimensions à chaque document de spec reçu avant que le développement commence. Partager le score avec toute l'équipe.
2. **Définir.** Démarrer un glossaire de projet dès la semaine 1, en session négociée avec le client. Y référencer chaque terme clé dans chaque spec.
3. **Structurer.** Transformer les specs prioritaires en format IF…THEN… complet (cas nominal + cas d'exception) avant le premier sprint de développement.
4. **Valider.** Organiser un workshop IF…THEN… client dans les deux premières semaines de chaque projet. En faire un livrable du plan de projet.
5. **Enregistrer.** Créer un registre d'ambiguïtés visible de toute l'équipe. Appliquer la règle de blocage : une ambiguïté non résolue bloque la story concernée lors du sprint planning.

Ces cinq engagements ne résolvent pas tous les problèmes de spécification. Mais ils créent les conditions dans lesquelles les autres solutions peuvent s'appliquer. Ils sont le sol. La Partie B est la construction.

Fin de la Partie A

La Partie B — *Spécifier pour construire juste : les pratiques du SDLC réconcilié* — est disponible dans le volume complémentaire du Livre 17.

Collection ToTAL — Réconciliation · 21 livres

L17 — Guide Jaune — Clarté et spécification · Partie A

Images et couverture par Soléane Bélanger

Léo Michel Bélanger, fondateur d'ORIGINALIUM